

Surname: Moqadam Mehr

Name: Aref

---

Title: Gesture Recognition via Spike-Convolutional Neural Networks (S-CNN)

---

Supervisors: Dr. Hadi Farahani and Dr. Saeed Reza Kheradpisheh

---

Degree: Master of Science

Subject: Department of Computer Science

Field: Algorithms and Computation Theory

---

Shahid Beheshti University

Faculty of Mathematical Sciences

Date: April 30, 2021

Number of pages: 60

---

Keywords: Spiking Neural Networks, Convolutional Neural Networks, Human Gesture Recognition, Dynamic Vision Sensor, Computer Vision, Machine Learning

---

### **Abstract**

Biological neurons use spikes to process and learn temporally dynamic inputs in energy and computationally efficient way. Unlike conventional Artificial Neural Networks, Spiking Neural Networks (SNN) tries to imitate this characteristic of the biological neurons to have advantages, such as energy efficiency. However, applying state-of-the-art gradient-based supervised algorithms to spiking neural networks is a challenge due to the non-differentiability of the activation function of spiking neurons. Employing surrogate gradients is one of the leading solutions to overcome this challenge. Although SNNs naturally work in the temporal domain, recent studies have focused on developing SNNs to solve static image categorization tasks. This paper employs a surrogate gradient descent learning algorithm to recognize twelve human hand gestures recorded by dynamic vision sensor (DVS) cameras.



Shahid Beheshti University  
Faculty of Mathematical Sciences  
Department of Computer Science

Master of Science Thesis

# **Gesture Recognition via Spike-Convolutional Neural Networks (S-CNN)**

by

Aref Moqadam Mehr

Supervisors

**Dr. Hadi Farahani and Dr. Saeed Reza Kheradpisheh**

April 30, 2021

*Dedicated to the passengers of Flight 752...*

*Attainment comes through knowledge and through good deeds...*

*and without knowledge one cannot even attain good deeds, for one must first know the good before one can perform it. Therefore, knowledge is indispensable in all circumstances in order to attain salvation. And when knowledge leads the way, it becomes the best thing a person can possess. In the beginning of creation it is not readily obtained; some of it is acquired without toil or contemplation — yet it is certainly the most important thing in whose acquisition one spends one's life. However, some knowledge comes without deliberation while some necessarily requires it; and that which is obtained through deliberation will require known premises upon which to deliberate, so that through deliberating upon the known, the unknown may through that deliberation become known. Not every unknown can be recognized through every known premise; rather, each unknown can only be recognized through the premise that is appropriate to it. And logic is that science through which the path from the unknown to the known is learned...*

*Therefore, logic is indispensable for the seeker of salvation.*<sup>1</sup>

---

<sup>1</sup>Introduction to the Logic treatise of *Dānishnāma-ye Alā'ī*, Sheikh al-Ra'īs Ibn Sīnā

## *Acknowledgements...*

At the outset, I consider it my duty to express my deepest gratitude and appreciation for the tireless efforts, relentless dedication, and wise guidance of my respected supervisor, Dr. Hadi Farahani. I also sincerely thank my esteemed advisor, Dr. Saeed Reza Kheradpisheh, who graciously agreed to advise this research and, throughout this journey, spared no cooperation or assistance. I extend my utmost gratitude to the distinguished professors in whose presence I had the honor of learning during my studies: Dr. Eslahchi and Dr. Ketanfaroush.

I also extend my deepest thanks to the Mathematics Department of the Faculty of Mathematical Sciences at Shahid Beheshti University, for providing suitable computational infrastructure and processing servers that gave us the opportunity to train more complex neural networks. Without such computational resources, the completion of this thesis would not have been possible.

I am also grateful to my dear and valued life partner, Ms. Zahra Noury, for her generous support in advancing this research and for her assistance in writing and proofreading this thesis.

And finally, I am forever thankful to my dear mother and father, who have always been by my side throughout this journey.

Aref Moqadam Mehr

April 30, 2021



# Contents

|                                                                              |          |
|------------------------------------------------------------------------------|----------|
| <b>Introduction</b>                                                          | <b>1</b> |
| <b>1 Spiking Neural Networks</b>                                             | <b>3</b> |
| 1.1 Brain Structure . . . . .                                                | 3        |
| 1.2 Spike Encoding Methods . . . . .                                         | 6        |
| 1.2.1 Rate-Based Spike Encoding . . . . .                                    | 6        |
| 1.2.2 Time-Based Spike Encoding . . . . .                                    | 6        |
| 1.3 Common Spiking Neuron Models . . . . .                                   | 7        |
| 1.3.1 Leaky Integrate-and-Fire Model . . . . .                               | 7        |
| 1.3.2 Integrate-and-Fire Model . . . . .                                     | 8        |
| 1.4 Training Spiking Networks . . . . .                                      | 9        |
| 1.4.1 Local Training Methods . . . . .                                       | 9        |
| 1.4.2 Binarization of Artificial Neural Networks . . . . .                   | 10       |
| 1.4.3 Conversion of Artificial Neural Networks to Spiking Networks . . . . . | 10       |
| 1.5 Supervised Learning Method . . . . .                                     | 10       |
| 1.5.1 Softening the Spiking Network . . . . .                                | 11       |
| 1.5.2 Using Differentiable Soft Spike Functions . . . . .                    | 11       |
| 1.5.3 Gradients in Probabilistic Models . . . . .                            | 12       |
| 1.5.4 Gradients in Rate-Coding Networks . . . . .                            | 13       |
| 1.5.5 Gradients in Single-Spike Networks (Spike-Time Coding) . . . . .       | 13       |
| 1.5.6 Surrogate Gradient Method . . . . .                                    | 14       |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>2</b> | <b>Problem Definition</b>                                     | <b>17</b> |
| 2.1      | Gesture Recognition . . . . .                                 | 17        |
| 2.2      | Literature Review . . . . .                                   | 19        |
| <b>3</b> | <b>Methodology and Research</b>                               | <b>25</b> |
| 3.1      | Data Preparation . . . . .                                    | 25        |
| 3.1.1    | Pixel-Based Image to Spike-Based Image Conversion . . . . .   | 26        |
| 3.1.2    | Spike Stream Framing . . . . .                                | 28        |
| 3.1.3    | Data Augmentation . . . . .                                   | 31        |
| 3.2      | Network Architecture . . . . .                                | 33        |
| 3.2.1    | Dense Layer . . . . .                                         | 34        |
| 3.2.2    | Convolutional Layer . . . . .                                 | 35        |
| 3.2.3    | Max-Pooling and Avg-Pooling Layers . . . . .                  | 35        |
| 3.2.4    | Not Using Synaptic Current Coefficient ( $\alpha$ ) . . . . . | 36        |
| 3.2.5    | Normalizing Output Spikes . . . . .                           | 36        |
| 3.2.6    | Initial Weights . . . . .                                     | 38        |
| 3.2.7    | Dropout Layer . . . . .                                       | 39        |
| 3.2.8    | Lateral Connections . . . . .                                 | 39        |
| 3.2.9    | Recurrent Connections . . . . .                               | 40        |
| 3.2.10   | Using Winner-Takes-All Mechanism . . . . .                    | 40        |
| 3.3      | Decision Making and Loss Function Calculation . . . . .       | 40        |
| 3.3.1    | Decision Making in the Output Layer . . . . .                 | 41        |
| 3.3.2    | Loss Function Calculation . . . . .                           | 43        |
| 3.4      | Implementation . . . . .                                      | 45        |
| 3.5      | Results . . . . .                                             | 49        |
| 3.5.1    | Complete Network Testing . . . . .                            | 49        |
| 3.5.2    | Frame-Based Image Generation Methods Testing . . . . .        | 49        |

---

|                       |                                                                    |           |
|-----------------------|--------------------------------------------------------------------|-----------|
| 3.5.3                 | Data Time Length Testing . . . . .                                 | 52        |
| 3.5.4                 | Output Type Testing . . . . .                                      | 52        |
| 3.5.5                 | Auxiliary Connections Testing . . . . .                            | 53        |
| 3.5.6                 | Comparison of Spiking Network with Artificial Neural Network . . . | 53        |
| 3.5.7                 | Conclusion . . . . .                                               | 54        |
| <br><b>Index</b>      |                                                                    | <b>57</b> |
| <br><b>References</b> |                                                                    | <b>57</b> |



# Introduction

The primary objective of this thesis is to analyze and investigate Spiking Neural Networks and to present a method for recognizing dynamic gestures using these networks. In recent years, neural networks have achieved remarkable advancements and are now applied to a wide variety of problems. One class of neural network is the Spiking Neural Network, whose neuron model bears greater resemblance to biological neurons. These networks possess high capabilities and have received significant attention in recent years.

In the first chapter, Spiking Neural Networks are introduced and their underlying principles are explained. In the second chapter, a summary of recent research in this field is reviewed. Finally, in the concluding chapter, the implemented method is described in detail and its results are discussed.

*Translator's note:* This thesis was originally written in Persian (Farsi). The English translation presented here was produced with the assistance of AI translation tools, specifically Anthropic's Claude and OpenAI's ChatGPT. While every effort has been made to maintain accuracy and terminological consistency, readers should be aware that the translation may contain imperfections inherent to AI-assisted translation.



# Chapter 1

## Spiking Neural Networks

### 1.1 Brain Structure

The objective of this thesis is to utilize Spiking Neural Networks for addressing machine vision challenges. As these networks draw inspiration from the biological neural system, acquiring knowledge about these systems can prove highly valuable. In essence, the nervous system of living organisms (the brain or the information processing system) comprises a complex intertwined assembly of neurons. These neural cells are also referred to as neurons. These cells in living organisms consist of a central nucleus, several input terminals<sup>1</sup>, which in this thesis we will call synapses, and an output terminal<sup>2</sup>. A simplified depiction of a biological neuron is shown in Figure 1.1. These networks function based on electrical pulses transmitted and operating between them. In other words, when the input voltage from the synapses reaches the expected threshold of the cell, the cell becomes stimulated and generates a Neural Spike at its output terminal. Subsequently, these pulses encounter the synapses of other cells, giving rise to a series of subsequent pulses that ultimately culminate in the intelligent operation of the brain.

The visual part of the mammalian brain exhibits a hierarchical structure Grill-Spector and Malach (2004). This signifies that it analyzes information through successive layers and transfers them from one layer to the next. These layers, situated in the Ventral Pathway of the Visual Cortex, have distinct responsibilities for image analysis at specific levels DiCarlo et al. (2012). Neurons along these hierarchical levels progressively engage

---

<sup>1</sup>These terminals are also referred to as synapses or dendrites.

<sup>2</sup>These terminals are also referred to as axons.

in the recognition of simple shapes such as lines and angles in the early regions of the visual cortex, eventually leading to the recognition of objects in the inferotemporal cortex [DiCarlo and Cox \(2007\)](#).

The major difference between Artificial Neural Networks and Spiking Neural Networks lies in the modeling of neurons. Although the structure of Artificial Neural Networks is inspired by the brains of living organisms [LeCun et al. \(2015\)](#), in reality, these networks do not bear much resemblance to the neural systems found in biological organisms because in these networks, the structure and model of neurons have been completely disregarded, and they have focused solely on emulating the interconnection of neurons and network structure. Artificial neurons have continuous output functions, whereas biological neurons have spike or impulse output functions. Research indicates that the mammalian brain spikes at a very slow rate. However, encoding information in the timing of spikes allows the same amount of spikes to be sufficient for information transmission. This enables our brain to be both highly efficient and low-power consuming [Kheradpisheh et al. \(2018\)](#). The interconnection between these neurons gives rise to more complex and powerful neural networks. These networks, which today are known as complete networks, feedforward networks, or multilayer networks, are typically composed of interconnected layers and possess significantly higher capabilities compared to individual neurons.

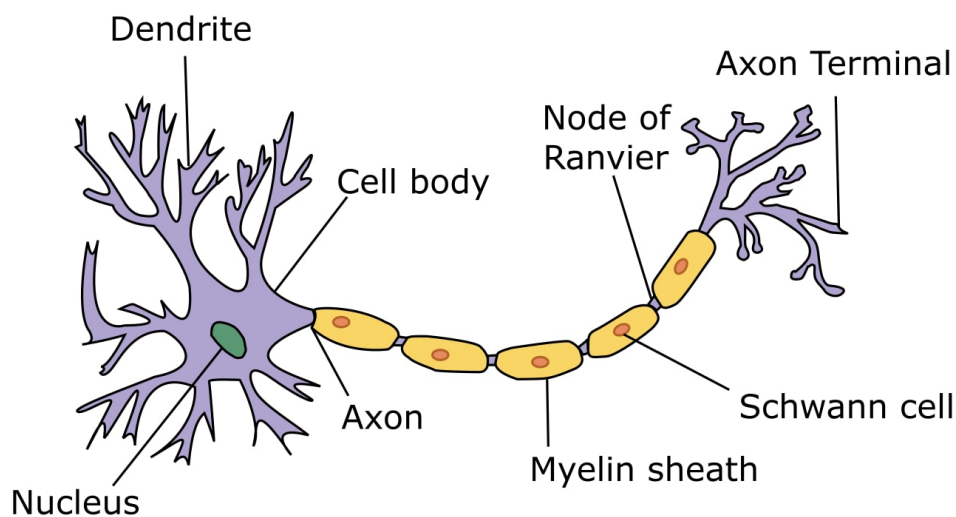


Figure 1.1: Simplified diagram of a neural cell in the brain of living organisms.

One of the subfields of neural networks is Spiking Neural Networks, which have received significant attention in recent years. These networks, compared to Artificial Neural

Networks<sup>3</sup>, have a closer resemblance to the biological brain. To gain a better understanding of the subject, we can compare conventional artificial neurons with biological neural cells.

The major difference between these two lies in how they generate output. Unlike artificial neurons, which have continuous outputs, the outputs of biological neural cells are in the form of electrical pulses. In fact, these cells receive electrical pulses from their input synapses, causing their potential to increase. Once their potential reaches a certain threshold tolerance, the neural cell emits an electrical pulse, resulting in the discharge of its potential. Spiking Neural Networks are also modeled based on this principle.

Therefore, perhaps the most important distinction between these two networks is that the definition of Spiking Networks is time-dependent. Spiking Networks are defined in the temporal dimension, and unlike Artificial Networks that examine the problem at a single moment and produce the output for that moment, Spiking Networks generate their outputs over time. For example, at time  $t_0$ , we start feeding input to the network. Although the network does not produce any output at this moment, after a time interval of  $k_1$  time units, the network generates its first spike. And after  $k_2$  time units have elapsed, the network has produced enough spikes for us to make decisions and draw conclusions about the output. This characteristic allows the accuracy of these networks to be adjustable according to requirements. In other words, if there is a pressing need to produce output, even if imprecise, we can make decisions earlier.

As mentioned earlier, these networks are inspired by the brains of living organisms, and due to the temporal property embedded in them, these networks can also be classified as Recurrent Networks as they share many similar properties and applications. In the following sections, we delve into further details of these networks. These details include methods for encoding data and spikes, as well as the dynamical model within neurons.

---

<sup>3</sup>Although the term "Artificial Neural Network" is not entirely accurate since both types of networks are artificial, in this thesis, "Artificial Neural Networks" refer to networks that have less resemblance to the human brain and have real-valued outputs, whereas "Spiking Neural Networks" refer to networks that have a closer resemblance to the human brain.

## 1.2 Spike Encoding Methods

In contrast to artificial neural neurons that have activation functions with real-valued outputs, Spiking Neurons have activation functions with pulse or spike outputs. These pulses have the same magnitude and duration for all neurons. Therefore, this value by itself cannot convey any information. Consequently, Spiking Networks operate in the temporal dimension to store information in the timing of spike generation. The manner in which this information is stored is referred to as the network's encoding method, which is one of the important and determining parameters of the network. Two general methods exist for encoding information in these neurons: rate-based spike encoding and time-based spike encoding.

### 1.2.1 Rate-Based Spike Encoding

In this method, information encoding is stored in the frequency or rate of spikes. In other words, the higher the frequency at which a neuron generates spikes, the more activity it has. In fact, the spike frequency in this method can be regarded as corresponding to the real-valued output of artificial neurons. One example is the conversion of ReLU neuron outputs to Spiking Neuron frequencies. It is evident that in this method, neurons can produce more than one spike, which consequently enables the network to have the capability to process data streams.

One advantage of this approach is that the spike propagation model is more similar to our brain compared to the spike timing method. For this reason, these models contribute more to the analysis of our brain's mechanisms. Additionally, these models have the capability to process sequential data without the need to reset the network.

### 1.2.2 Time-Based Spike Encoding

In contrast, the spike timing method transfers information through the timing of spike propagation. This means that the timing of the first spike indicates the neuron's activity. The closer this timing is to zero, the higher the neuron's activity, and the larger this timing, the lower the activity level. If a neuron does not produce any spikes, it indicates zero neuron activity.

It is evident that in this method, only the first spike is considered, and therefore, subsequent spikes have no informational value. For this reason, in implementations of this method, only one spike is considered for each neuron, and each neuron can produce a maximum of one spike.

These models are simpler compared to spike rate models and produce fewer spikes. Therefore, it can be said that these models have more optimized energy consumption. However, on the other hand, these models do not inherently possess the capability to process data streams. To address this issue, we need to consider a mechanism for resetting the network and neurons. For example, after solving each test sample or after analyzing each signal, we reset the potentials of neurons to zero using a specific signal so that they are ready for subsequent executions.

### 1.3 Common Spiking Neuron Models

In this section, we investigate the dynamical model within neurons and examine how their internal potential functions.

#### 1.3.1 Leaky Integrate-and-Fire Model

The most common approach for modeling Spiking Neurons is the Leaky Integrate-and-Fire (LIF) model [Gerstner et al. \(2014\)](#). These neurons bear a strong resemblance to our brain's functioning. Equation 1.1 represents the model of these neurons in the absence of spike-generating mechanisms, which leads to the Membrane Potential returning to its initial value.

$$\tau_{mem} \frac{dU_i}{dt} = RI_i - (U_i - U_{rest}) \quad (1.1)$$

In this equation,  $U_i$  represents the current Membrane Potential of the neuron, and  $U_{rest}$  denotes the Resting Potential of the neuron. The resting potential is the value that the neuron reaches when it has no input or output, or after spiking. The term  $\frac{dU_i}{dt}$  indicates the instantaneous changes in the Membrane Potential, which is controlled by the constant  $\tau_{mem}$ . This constant actually represents the leakage of the neuron's potential.  $I_i$  represents the Input Current, and  $R$  represents the resistance of that input. Moreover,

to model the neuron's Input Current, we utilize the power-tracking method [Neftci et al. \(2019\)](#). Equation 1.2 is employed for this model.

$$\frac{dI_i}{dt} = -\frac{I_i(t)}{\tau_{syn}} + \sum_j W_{ij} S_j(t) \quad (1.2)$$

In this equation, similar to the previous one, we aim to model the changes in the current value. This value depends on the previous current value, which is controlled by the constant  $\tau_{syn}$ , and the input spikes of the neuron, each weighted by its corresponding weight  $W$ . We know that the Membrane Potential of neural neurons, after reaching the threshold tolerance, triggers the neuron to generate a spike and causes the potential value to decrease. Therefore, by adding the spike model to Equation 1.1, we can obtain Model 1.3.

$$\frac{dU_i}{dt} = -\frac{1}{\tau_{mem}}(U_i - U_{rest}) + RI_i + S_i(t)(U_{rest} - \theta) \quad (1.3)$$

In this equation, the variable  $S_i(t)$  represents whether the target neuron has an output spike at time  $t$  or not. This value is defined by Equation 1.4. The parameter  $\theta$  represents the threshold tolerance of the neuron.

$$S(t) = \begin{cases} 1 & \text{if } U_i \geq \theta \\ 0 & \text{otherwise} \end{cases} \quad (1.4)$$

To simplify the computations, Equations 1.1 and 1.2 can be written in a discrete form as follows:

$$I_i[n+1] = \alpha * I_i[n] + \sum_j W_{ij} S_j[n] \quad (1.5)$$

$$U_i[n+1] = \beta * U_i[n] + I_i[n] - S_i[n] \quad (1.6)$$

### 1.3.2 Integrate-and-Fire Model

This model, which is also referred to as Integrate-and-Fire (IF), is completely similar to the LIF model, and the only difference is that these models have no leakage. To derive

the equation for this model, it is sufficient to set the value of  $\beta$  to one in Model 1.6, thus disregarding neuronal leakage. These models employ a simpler mechanism for information processing, and one of their applications is use in spike timing encoding models.

## 1.4 Training Spiking Networks

Due to the distinct dynamics of Spiking Neurons, their dependence on the temporal dimension, and the non-differentiability of the spike function, traditional training methods cannot be applied to these neurons. Consequently, specific approaches have been developed to address this challenge, which we will examine in the following.

### 1.4.1 Local Training Methods

One of the most common local training approaches for Spiking Neural Networks is the Spike-Timing Dependent Plasticity (STDP) method. This method, which falls under the category of unsupervised learning techniques, bears a strong resemblance to learning mechanisms in the brain. In this approach, the training of each neuron depends on the input activities before and after the moment it spikes. Specifically, the Synaptic Weights of inputs that were activated before the spike moment are strengthened, while the weights of inputs that were activated after the spike moment are weakened. The mathematical model for this method is represented by the following equations:

$$\Delta W_{ij} = \begin{cases} a^+ w_{ij} (1 - w_{ij}) & \text{if } t_j - t_i \leq 0 \\ a^- w_{ij} (1 - w_{ij}) & \text{if } t_j - t_i > 0 \end{cases} \quad (1.7)$$

In this equation,  $i$  and  $j$  represent the Post-Synaptic Signal (the signal related to the output spikes of the neuron) and the Pre-Synaptic Signal (the signal related to the input spike trains of the neuron), respectively. The weights  $a^+$  and  $a^-$  correspond to the positive and negative learning rates of the neuron, respectively, which are determined as input parameters to the system. These two values actually control the neuron's modifications. The term  $w_{ij}(1 - w_{ij})$  ensures that the weight values remain within the range of zero to one. In fact, this term drives the magnitude of changes towards zero at the boundaries of the zero-to-one range.

One of the advantages of STDP methods is their unsupervised nature. Therefore, these methods can attempt to learn patterns without having explicit labels for the data. However, on the other hand, this inability to learn labels can also lead to challenges. For example, the network may try to learn patterns that do not contribute significantly to problem-solving. Additionally, another drawback of this approach is its inefficiency in networks with deeper layers.

#### 1.4.2 Binarization of Artificial Neural Networks

In this approach, the only modification made to Artificial Neural Networks is their activation function, such that their activation function is changed to a spike function. As a result, these networks exhibit some properties of Spiking Networks, including that information transmission between neurons will be binary, which enhances their efficiency and reduces the required memory.

#### 1.4.3 Conversion of Artificial Neural Networks to Spiking Networks

Another existing approach is the conversion of Artificial Neural Networks to Spiking Neural Networks. For this purpose, we first train the network using available data. Then, we apply predefined constraints to transform the network into a Spiking Network. For example, a transformation function can be defined to convert the real-valued function output to spike timing. Subsequently, we select the network weights in such a way that the Spiking Neural Network produces output corresponding to the Artificial Neural Network.

An important and contemplative point in this regard is that not all elements of Artificial Networks are convertible to Spiking Neural Networks. For instance, layers such as Average-Pooling or activation functions other than ReLU are not convertible to spiking elements because these elements have a real-valued nature.

### 1.5 Supervised Learning Method

In previous sections, the supervised learning method has been mentioned. In summary, in this approach, we need to determine the loss function in such a way that the error on the desired output is minimized. Then, we calculate the error generated by each neuron

in order to adjust the neuron weights in a way that minimizes this error and consequently produces the desired output. The challenge in Spiking Networks lies in the fact that the activation function of Spiking Neurons, the spike function, is non-differentiable. In fact, its derivative is zero at all points, and it is undefined at the moment when the function activates and generates a spike. Therefore, we cannot use conventional methods for optimizing the network. In Section 3.3.2, we will examine this issue and provide a precise definition of the loss function. To address the non-differentiability of the function, four general strategies have been proposed:

1. Complete implementation of biological processes and training through these methods, for example, using local training methods such as STDP.
2. Conversion of Spiking Networks to Artificial Networks.
3. Softening the Spiking Network and using soft spikes.
4. Using Surrogate Gradients for spike functions.

In previous sections, we briefly discussed the first and second points. In the following, we will examine the third and fourth points.

### 1.5.1 Softening the Spiking Network

In essence, in the softening method of the Spiking Network, we use functions and methods that ensure the differentiability of the network. For this purpose, four methods have been proposed, which include: using differentiable soft spike functions, gradients in probabilistic models, gradients in rate-coding networks, gradients in single-spike networks (time-coding), and the Surrogate Gradient method.

### 1.5.2 Using Differentiable Soft Spike Functions

This method can be implemented on all Spiking Networks. In this approach, instead of using a hard spike function, we use soft spike functions. Unlike the hard spike function, which has a discontinuous value and is zero everywhere except at  $U = \theta$ , the soft spike function has a continuous value and is differentiable at all points. The value of this function smoothly approaches the point  $\theta$ . An example of such functions is shown in Figure 1.2.

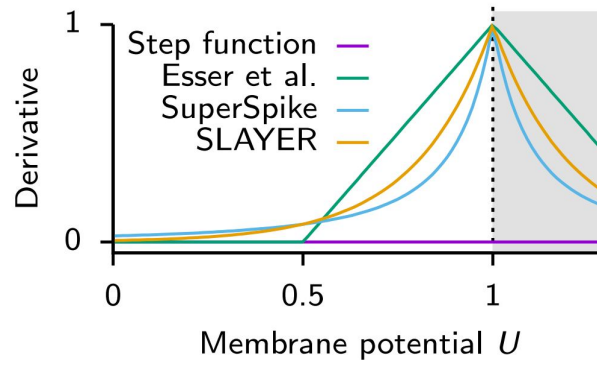


Figure 1.2: The spike function (first case) and its comparison with examples of soft spike functions. Source [Neftci et al. \(2019\)](#)

The advantage of this method is that after changing the function, it becomes easy to use training methods for Recurrent Networks, such as BPTT<sup>4</sup> or RTRL<sup>5</sup>, to train these networks. However, the problem with this approach is that the network transitions from the spiking state, and the neurons no longer possess binary properties. As we know, having binary output is one of the most essential characteristics of Spiking Networks. Therefore, it can be said that these networks no longer have some of the properties of Spiking Networks and more closely resemble artificial Recurrent Networks.

### 1.5.3 Gradients in Probabilistic Models

The next method is using probabilistic models to compute gradients. In this approach, the spike function is defined probabilistically. Therefore, the gradient of the function can be easily defined based on the mathematical expectation, while the function still has binary output. Single-layer and multilayer networks based on this model have been investigated in studies [Gardner et al. \(2015\)](#), [Brea et al. \(2013\)](#), and [Mostafa and Cauwenberghs \(2018\)](#). The only issue with this method is that the noise introduced into the system during probabilistic spike sampling causes the spike function to soften and become differentiable, and on the other hand, it sometimes makes the model difficult to optimize, which affects its convergence.

<sup>4</sup>Back Propagation Through Time

<sup>5</sup>Real-Time Recurrent Learning

### 1.5.4 Gradients in Rate-Coding Networks

Another common approach to obtaining gradients in Spiking Networks is utilizing the concept of rate-coding. In this method, the factors for spike generation in Spiking Neurons are the input spike volume of that neuron. Therefore, it can be said that there exists a relationship between the number of input and output spikes. We can plot this relationship in the form of a current-spike or f-I curve for the neuron, such that on one axis we plot the input current to the neuron and on the other axis we plot the output spike rate. In these cases, the derivative of the f-I curve can be a very suitable option for neuron optimization. Various research has been conducted in this area, including [Hunsberger and Eliasmith \(2015\)](#) and [Neftci et al. \(2017\)](#). In each of these cases, rate coding was employed and excellent results were achieved on datasets such as CIFAR-10 and MNIST. Additionally, in [Lee et al. \(2016\)](#), which presents a deep network, training is based on this derivative method. In this approach, a low-pass filter was applied to the input spikes to improve the results.

Rate-coding gradient methods can be very effective and produce good results. However, these methods also have other challenges. For example, the accuracy of these models can significantly decrease under certain conditions because calculating the f-I curve requires averaging the number of input and output spikes. This averaging process needs some time to pass from the beginning of processing to minimize the noise generated during this process.

### 1.5.5 Gradients in Single-Spike Networks (Spike-Time Coding)

In another attempt to optimize Spiking Networks without introducing unwanted noise and without using rate-coding methods, the use of single-spike coding methods has been explored. In these networks, where each neuron generates only one spike, each spike carries a significant amount of information. The main idea for this approach was proposed in [Bohte et al. \(2002\)](#). In this method, the spike timing in the hidden layers of the network is calculated, and a linear function is considered for this value. Therefore, the derivative of this function can be easily calculated. However, this approach has its own limitations. The first and most important drawback is that this method is only applicable to networks that are single-spike. Additionally, another issue with these networks is neurons that produce

spikes and are silent. It is not possible to compute the gradient value for these neurons, and we need to implement another method for these neurons.

### 1.5.6 Surrogate Gradient Method

Surrogate Gradient methods provide a suitable solution to the issue of the discontinuity of the activation function in Spiking Neurons. The main philosophy of this approach is to propose a method that softens the gradient instead of changing the function and gradient, resulting in a continuous gradient value. The more important issue is that this gradient softening only applies during the parameter optimization stage and has no effect during the network's execution phase. Another significant advantage of this method is that it can train the network without the need to specify encoding methods and other parameters. In fact, this method does not need to know these parameters. The Surrogate Gradient method has the capability to utilize techniques like BPTT, similar to conventional gradient descent methods.

To date, numerous studies have employed Surrogate Gradient methods to train Spiking Networks using the BPTT technique. This approach involves a slight modification to the network, where in each iteration of the training algorithm, the gradient value of another function is used instead of the gradient of the spiking function. This method can even be applied in modern tools that utilize automatic differentiation techniques. One of the earliest implementations of these methods was presented in [Bohte \(2011\)](#), where the gradient of the spiking function was replaced by the gradient of truncated polynomials, resulting in the derivative of the ReLU function. Moreover, research in [Courbariaux et al. \(2016\)](#) and [Esser et al. \(2016\)](#) also drew inspiration from the Surrogate Gradient method to train larger networks with convolutional layers with spiking functions. In [Zenke and Ganguli \(2018\)](#), a three-factor learning rule was proposed that uses the negative part of the sigmoid function to create the Surrogate Gradient. Additionally, in [Bellec et al. \(2018\)](#), a network with temporal properties was successfully trained using the same negative part of the sigmoid function. In this study, the author discovers that this method can be used to train a convolutional network with long short-term memory layers. Finally, in [Shrestha and Orchard \(2018\)](#), an exponential function was used for the Surrogate Gradient, achieving very good results compared to previous neuromorphic hardware approaches.

---

In summary, numerous studies have explored the use of Surrogate Gradient methods, employing various activation functions and different replacement functions, investigating networks with diverse architectures. Although all these activation functions are different, they all share one common characteristic, which is that they all uniformly increase in value relative to the spike point.



# Chapter 2

## Problem Definition

The selected problem for this thesis revolves around dynamic gesture recognition using spiking neural networks. In this section, we examine the problem statement and the objectives underlying its resolution.

### 2.1 Gesture Recognition

Gesture recognition is one of the branches of computer vision, wherein machine learning methods are employed to identify human gestures. To comprehend the nature of this problem, it is essential to initially define the term "gesture." In this context, a gesture, which is the translation of the English word "Gesture," refers to the physical position of the body that can convey information about its internal state. The examined physical body in this context is the human, and the purpose of these gestures is to express information or requests. A simple example of such gestures is a smile, indicating feelings of joy and enthusiasm in an individual, serving as a means to convey meaningful information in interpersonal interactions. We are constantly engaged in interaction and information transfer through these gestures.

Recognition of these gestures can find applications in various fields. One example of this problem can be human motion analysis, and solving this problem can be useful for medical purposes. One such example is applications that attempt to identify problems in a person's musculoskeletal system through skeletal structure and motion analysis [Neto et al. \(2015\)](#) and [Mozaz et al. \(2006\)](#). Another example of the application of these problems can be in human-robot interaction, which itself forms a very large branch. In such problems,

we seek for the robot to be able to understand human movement. For example, it should recognize and execute a command that the human subject intends [Mitra and Acharya \(2007\)](#) and [Ren et al. \(2011\)](#).

Next, we examine dynamic gestures. These gestures refer to movements where each segment, in isolation and at rest, may have a different meaning or may not convey a specific meaning at all, but the cohesive set of these gestures conveys a unified meaning. For instance, waving one’s hand in the air can signify a farewell gesture.

In general, gesture recognition can have numerous applications. Their most significant application lies in human-robot interaction. In other words, these gestures can be one of the interfaces for our engagement with the world of artificial intelligence. These interactions can range from simple examples such as controlling household devices to more complex scenarios, such as the recognition and analysis of human emotions by artificial intelligence.

Recognizing static gestures that do not involve motion is relatively simpler compared to dynamic gestures, as the temporal dimension is not considered in these gestures. However, recognizing dynamic gestures can be much more valuable than static gestures because dynamic gestures can encapsulate significantly more information.

In this thesis, two datasets have been employed. The first dataset was curated by IBM, where individuals perform a specified set of dynamic gestures in front of a Dynamic Vision Sensor (DVS). In this dataset, 29 different participants execute 11 dynamic gestures under three distinct lighting conditions. The lighting conditions and dynamic gestures are outlined in [Tables 2.1 and 2.2](#). The distinctive feature of this dataset lies in the varying lighting conditions, as these conditions can significantly impact the spike generation pattern in the DVS sensor. For instance, spikes generated in natural light conditions are nearly uniform, whereas under lamp light, the sensor produces spikes with a pattern corresponding to the lamp frequency. This dataset is named "IBM-DVSGesture" [Amir et al. \(2017\)](#). A sample from these data is illustrated in [Figure 2.1](#).

The second dataset, named "NavGesture," comprises fewer data compared to the previous dataset. For data collection, a DVS sensor is installed on a mobile phone, observing the subject from this perspective, and the subject issues gesture commands to the mobile phone. The objective of this research is to advance human interaction with mobile phones.

| ID | Gesture Label               |
|----|-----------------------------|
| 1  | hand clap                   |
| 2  | right hand wave             |
| 3  | left hand wave              |
| 4  | right arm clockwise         |
| 5  | right arm counter clockwise |
| 6  | left arm clockwise          |
| 7  | left arm counter clockwise  |
| 8  | arm roll                    |
| 9  | air drums                   |
| 10 | air guitar                  |
| 11 | other gestures              |

Table 2.1: Table of dynamic gestures in the IBM-DVSGesture dataset

| ID | Lighting Condition |
|----|--------------------|
| 1  | Fluorescent        |
| 2  | LED                |
| 3  | Natural Light      |

Table 2.2: Table of lighting conditions in the IBM-DVSGesture dataset

A notable aspect of this research is that it exists in two separate parts. In the first part, the subject is seated and stationary, while in the second part, the subject is in motion. This movement introduces environmental noise due to the camera’s motion [Maro et al. \(2020\)](#). The results of these investigations are discussed in Section 3.5.

## 2.2 Literature Review

The IBM-DVSGesture dataset was extracted from the paper [Amir et al. \(2017\)](#). In this research, the investigators attempted to solve the dynamic gesture recognition problem using a spiking neural network implemented on neuromorphic hardware. As mentioned earlier, this dataset is obtained by a DVS sensor. Furthermore, the implementation code in this study runs on the TrueNorth hardware, designed and developed by IBM. The structure of the spiking neural network employed in this research consists of four main

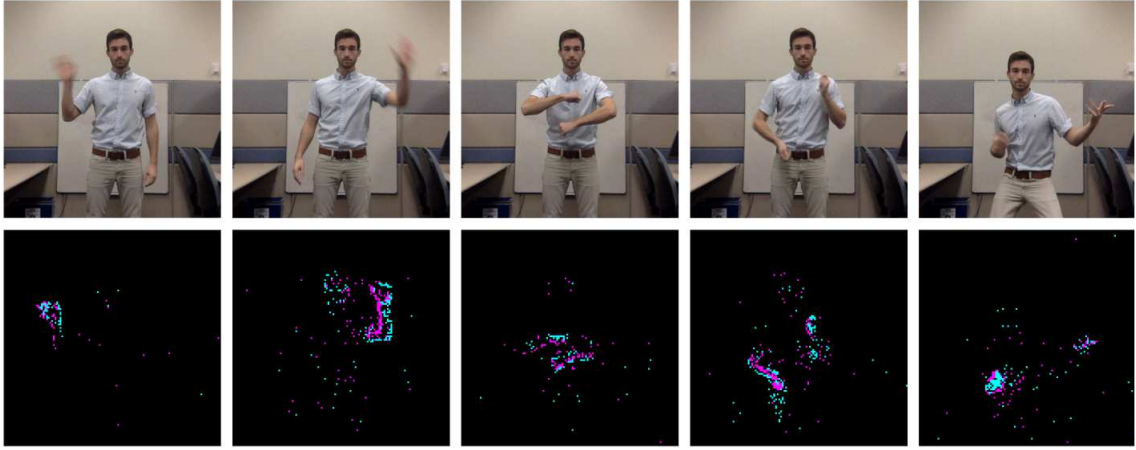


Figure 2.1: In this image, samples from the IBM-DVSGesture dataset are shown. The upper images were captured with a conventional pixel-based camera, and the lower images show the same subject captured with a DVS sensor. Source: [Amir et al. \(2017\)](#)

components, which are briefly elucidated below.

The first section of the network is a temporal filter layer responsible for maintaining consecutive snapshots of input data for a specified duration. This layer comprises  $k$  filters. Initially, the first filter stores the first series of input data. Subsequently, in the next time cycle, the second layer copies the values from the first layer, and then the first layer reads the second series from the input. This process continues, and information moves from the first filter to the last temporal filter. In essence, the task of this filter is to generate a form of short-term memory. It is noteworthy that all these layers are simultaneously connected to the convolutional layers.

The second component consists of convolutional layers. These layers are similar to convolutional layers in artificial neural networks, with the difference that a binary function is used instead of the ReLU activation function. Additionally, the weights of these layers are ternary, meaning they can take on values of zero, one, or negative one. Training in these layers occurs offline and outside the neuromorphic hardware. A form of backpropagation algorithm is employed for weight training. In this algorithm, a copy of the network weights is maintained in integer form, and the backpropagation and optimization operations are performed on these weights. Subsequently, these weights are converted into ternary weights. The network utilized in this study comprises 16 convolutional layers.

The third component is the winner-takes-all layer. Positioned after the convolutional

layers, this layer employs a mechanism that reduces the potential of neighboring neurons following the spike of a neuron. Consequently, after a neuron spikes, the likelihood of neighboring neurons spiking becomes very low. This layer also serves as the classification layer of the network, producing one of the classes as output. The final part is the sliding window filter, which smoothens the output of the third layer. The implementation method is averaging over the last  $n$  frames.

The research conducted in [Kheradpisheh et al. \(2018\)](#) demonstrated that it is possible to achieve acceptable results comparable to artificial neural networks using spiking neural networks. The network structure begins with a temporal encoding layer. This layer, of the Gaussian difference type, is responsible for converting pixel values of an image, representing the light intensity in those pixels, into spikes. This transformation occurs such that the higher the light intensity value in a pixel, the earlier that pixel spikes, and the lower the light intensity value, the later the spike occurs or does not occur at all.

After the encoding layer, there are several convolutional layers. In the problem discussed in this research, three convolutional layers are used, and after each layer, a pooling layer is placed. Finally, after the convolutional layers, there is a classification layer that detects the result of the convolutional layers and produces the output for the problem. It is worth noting that neurons in the convolutional layers are of the IF type, meaning their internal voltage does not decay over time, and they only accumulate the input current values.

The type of encoding used in this study is spike-time encoding. This means that in the first layer, after image spikes are generated, each input neuron produces at most one spike. Then, the convolutional layers collect these spikes, and after their internal voltage surpasses a certain threshold, they generate output spikes and do not produce any more spikes. In other words, the convolutional layers immediately produce spikes once their internal potential exceeds the tolerance threshold, and they no longer produce additional spikes. Similarly, in the pooling layers, neurons generate spikes after receiving the first input spike and ignore the remaining input spikes.

Another mechanism employed in this study is the winner-takes-all mechanism. This mechanism ensures that in each region of the convolutional layer, regardless of the feature map, if a spike occurs, no neuron in its vicinity can generate spikes. This ensures that the

network learns important and common features. We will delve into this in more detail in subsequent sections. The training in this network is performed layer-by-layer using the STDP method, and by layer-by-layer, we mean that the training of each layer begins after the completion of the training of the previous layer and is performed in an unsupervised manner.

In [Kheradpisheh et al. \(2016\)](#), deep convolutional networks were employed, but with the distinction that supervised learning was not utilized in this research. Instead, researchers used the STDP method for network training. [Gyöngyössy et al. \(2019\)](#) also attempted to solve the gesture recognition problem. Although this research used static gestures, it implemented spiking neural networks trained with the STDP method. In this research, IF neurons that generate spikes only once were used, and this network is not convolutional.

In the paper [Zenke and Ganguli \(2018\)](#), a method for training IF neurons has been introduced, allowing for the supervised training of multilayer networks. This method is based on the surrogate gradient method, where the Hebbian Three-Factor Rule is employed to train neuron weights. This method operates such that neurons whose polarity changes receive corresponding training signals with a predefined learning coefficient. This operation occurs during execution and does not require error backpropagation. In [Hunsberger and Eliasmith \(2015\)](#), a similar approach for training LIF neurons is presented.

Conventional and simple methods transmit training signals based on neuron activity levels. These methods often lose their efficiency as the network size increases and more layers are added. In the paper [Lillicrap et al. \(2016\)](#), a novel technique for backward transmission of training signals has been introduced, focusing solely on neurons that played a role in error generation. This paper was tested on the MNIST dataset and was able to reduce the training error of the tested network from 7% to 2% under identical conditions using only this method.

The same problem was addressed in the paper [Mostafa et al. \(2018\)](#) using a different approach. In this research, which was conducted on the CIFAR-10 dataset, each neuron calculates its training error directly using the network's final error and employing the Random Auxiliary Classifiers mechanism. Each neuron is trained using this error independently of other neurons and layers. One advantage of this method is that layers can

be trained simultaneously.

The paper [Lee et al. \(2016\)](#) addresses the challenge of non-differentiability in the spike function. In this paper, instead of deriving the spike function for computing the training signal, the derivative of the internal neuron potential is used. Since this value has a direct relationship with the neuron’s activity and the values of output spikes, and moreover, this value is continuous except at the spike occurrence moments, it serves as a suitable replacement for the spike function derivative. However, it is worth noting that this value has a discontinuity at the spike moment, which can be considered as a disturbance in the system and will not significantly impact the overall system performance. This paper was implemented on the MNIST dataset.

In [Nøkland \(2016\)](#), the backpropagation mechanism is entirely set aside, and instead, the Feedback Alignment method is employed. This method, which bears closer resemblance to biological neural networks, directly transmits the training error to layers and neurons through random connections and trains the network. This paper was implemented on the MNIST and CIFAR datasets.

Similarly to the network employed in [Mostafa and Cauwenberghs \(2018\)](#), a probabilistic neuron model is utilized. For training this model, instead of using backpropagation methods, the winner-takes-all mechanism is applied. In this mechanism, the neuron that produces spikes earlier is considered the winner, and the potentials of neurons in proximity to this neuron decrease. Consequently, the likelihood of these neurons spiking in subsequent cycles diminishes. As a result, neurons attempt to learn differences that have greater spatial separations. This study was implemented on the MNIST dataset.

In biological neural networks, neural signals may reach their destination with a delay depending on the physical distance between neurons. One method to enhance the efficiency of spiking neural networks is to implement this delay in neural signals. In the paper [Shrestha and Orchard \(2018\)](#), the simulation coefficient for the delay of neural signals is optimized concurrently with the network weights, demonstrating potential improvements in the network’s capabilities. This research utilizes a convolutional architecture for learning from the MNIST, TIDigits, and DVS-Gesture datasets. The modeling of neurons employs a probabilistic spiking neuron model, validated and examined on neuromorphic hardware.

One of the important architectures in neural networks is recurrent neural networks.

Research in papers [Huh and Sejnowski \(2017\)](#) and [Brea et al. \(2013\)](#) focused on recurrent neurons. In the neuron model presented by the paper [Brea et al. \(2013\)](#), recurrent neurons organized in a complete network undertake the learning process. In this method, a statistical model of spiking neurons is used, and neurons are trained by minimizing the upper bound of the Kullback-Leibler divergence. Additionally, the paper [Huh and Sejnowski \(2017\)](#) introduces a method for training and optimizing recurrent neurons based on the surrogate gradient method. In this paper, instead of the spike function, the training signal is computed from the potentials and input currents of neurons, and neurons are trained. Similarly, in [Bellec et al. \(2018\)](#), architectures with recurrence are employed, but with the distinction of implementing the LSTM mechanism and training it using the BPTT method.

Most of the aforementioned papers incorporate spike rate encoding. In the paper [Mostafa \(2017\)](#), a method for training networks based on spike-time encoding during execution is presented. In this paper, a relationship between input and output currents of neurons is established, which is differentiable everywhere, and the training signal can be calculated based on these values. This paper was also implemented on the MNIST dataset.

In the paper [Courbariaux et al. \(2016\)](#), a binary artificial neural network and a training method for it are introduced. Although these networks lack many advantages of spiking neural networks, due to their use of binary operators, they are much more efficient in terms of required memory and processing power compared to traditional artificial neural networks. Furthermore, due to their non-spiking nature, they can be easily simulated on GPUs. In this paper, which was implemented on the MNIST, CIFAR-10, and SVHN datasets, a method for calculating neuron gradients during network execution is also presented.

## Chapter 3

# Methodology and Research

In this research, we utilized two sources as the code base for this project: the SpyTorch project <sup>1</sup> and the S2Net project <sup>2</sup>.

The SpyTorch project has been developed on the Fashion-MNIST dataset. This dataset consists of 48 by 48 pixel grayscale images showcasing various types of clothing. Since this network operates on pixel-based image data, it utilizes a pixel-to-spike transformation layer, translating pixel-based images into spike-based representations. Additionally, this network employs spike rate coding. It's worth noting that this project exclusively implements fully connected layers and does not incorporate convolutional layers.

In the subsequent step, we examined the S2Net project, which is focused on spike-based audio data. In this project, simple convolutional layers have been implemented. However, based on the conducted investigations, this network alone is not capable of learning dynamic gesture data. The first step requires increasing the dimensionality of the convolutional layers. Moreover, this network lacks layers such as Batch Normalization, Dropout, Pooling, and others.

### 3.1 Data Preparation

In the initial steps of processing, data needs to be preprocessed and prepared. This stage is common in many neural networks, particularly those working with images. Often, this stage involves tasks such as cleaning the image, removing additional noise, resizing

---

<sup>1</sup><https://github.com/fzenke/spytorch>

<sup>2</sup><https://github.com/romainzimmer/s2net>

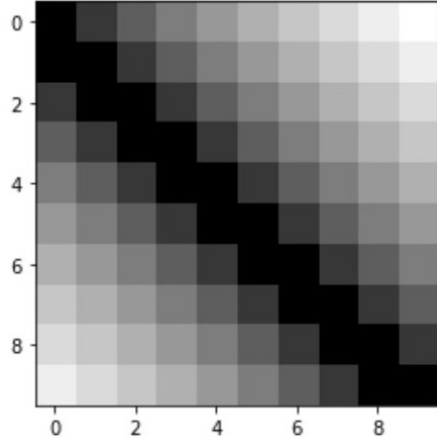
the image, etc. However, since the data used in this thesis is in the form of spikes, the data preparation stage is slightly different. The most crucial part of this stage is transforming the data from a spike stream into a set of frame-based spikes. The following provides more details on this process.

### 3.1.1 Pixel-Based Image to Spike-Based Image Conversion

As mentioned earlier, the structure of this project is based on the research presented in Neftci et al. (2019). However, the main difference in our work lies in the dataset we have utilized, which is composed of a spike stream. This is in contrast to the referenced research, which employs images based on pixels. Given that the network used is of the spiking type, it is necessary to convert image data into spikes. This task is performed by the transformation layer at the beginning of the network. The functioning of this layer can be observed in the following equation:

$$T(x) = \begin{cases} T_{max} & \text{if } x < threshold \\ \tau \log(\frac{x}{x-threshold}) & \text{otherwise} \end{cases} \quad (3.1)$$

In this equation, the function  $T$  represents the numerical mapping of pixel intensity to spike time. A crucial parameter in this equation is the *threshold* value. This value indicates the minimum pixel intensity required to allow spike generation. Values below this threshold do not produce spikes. In other words, a spike is generated at time  $T_{max}$ , which is effectively a time the network never reaches, and processing stops before reaching it. The next parameter is the value of  $\tau$ , which is the membrane time constant of the LIF neuron, determining when the neuron charges. Increasing this value causes spikes to occur more towards the end of the time spectrum, while decreasing it brings spikes closer to time zero. Finally,  $T_{max}$  is the maximum number of cycles the network operates. The network never reaches time  $T_{max}$ , and it stops before that. An example of the pixel value and its spike time is visible in Figure 3.1. It is worth mentioning that the value  $x$  before this equation needs to be normalized to a range between zero and one to ensure different image encodings do not adversely affect this equation. Since the data used in this thesis is in the form of spikes, there was no need to add this data transformation layer in this thesis.



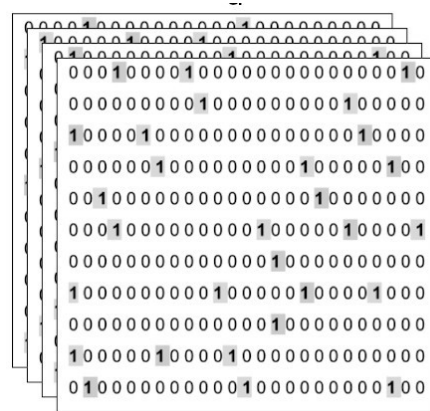
(a) Pixel-based image

```
[255, 199, 161, 130, 103, 79, 57, 36, 17, 0]
[255, 255, 199, 161, 130, 103, 79, 57, 36, 17]
[199, 255, 255, 199, 161, 130, 103, 79, 57, 36]
[161, 199, 255, 255, 199, 161, 130, 103, 79, 57]
[130, 161, 199, 255, 255, 199, 161, 130, 103, 79]
[103, 130, 161, 199, 255, 255, 199, 161, 130, 103]
[79, 103, 130, 161, 199, 255, 255, 199, 161, 130]
[57, 79, 103, 130, 161, 199, 255, 255, 199, 161]
[36, 57, 79, 103, 130, 161, 199, 255, 255, 199]
[17, 36, 57, 79, 103, 130, 161, 199, 255, 255]
```

(b) Numerical values of pixel light intensity

```
[12, 16, 20, 26, 34, 47, 74, 167, -, -]
[12, 12, 16, 20, 26, 34, 47, 74, 167, -]
[16, 12, 12, 16, 20, 26, 34, 47, 74, 167]
[20, 16, 12, 12, 16, 20, 26, 34, 47, 74]
[26, 20, 16, 12, 12, 16, 20, 26, 34, 47]
[34, 26, 20, 16, 12, 12, 16, 20, 26, 34]
[47, 34, 26, 20, 16, 12, 12, 16, 20, 26]
[74, 47, 34, 26, 20, 16, 12, 12, 16, 20]
[167, 74, 47, 34, 26, 20, 16, 12, 12, 16]
[-, 167, 74, 47, 34, 26, 20, 16, 12, 12]
```

(c) Spike time for each point of the image



(d) Spike matrix over time

Figure 3.1: Figure 3.1a shows an example of a pixel-based image. Figure 3.1b displays the pixel values of a small portion of the image. It should be noted that in uint8 encoding, pixel values range from 0 to 255. A value of 0 represents black (pixel off) and 255 represents white (pixel on). Figure 3.1c shows the converted image to spike time. Each element of the matrix specifies the time at which the pixel spikes, and values of *inf* indicate that the pixel does not produce a spike during the network's operation time. Figure 3.1d shows the spike value of pixels, which in each network cycle can take a value of one or zero, indicating whether a spike was produced in that cycle or not.

### 3.1.2 Spike Stream Framing

As mentioned in the previous section, data processing in the code [Neftci et al. \(2019\)](#) is frame-based. We also used frame-based processing in our project. Although frame-based processing is not suitable for implementation on neuromorphic hardware and the main purpose of using DVS sensors is to benefit from the sparsity property of the data, in this thesis, we disregarded this capability and converted and processed this data in matrix form, in a structured format that we call frame-based spike images. The reason for this decision is that simulating networks that work with structured data on GPUs, and generally on von-Neumann architectures, will be much simpler, and their processing will be significantly faster. For this purpose and also due to the lack of access to neuromorphic processors, we performed this conversion to both increase the processing and training speed of the network and to be able to develop the system in a shorter time.

Therefore, the first step in data preprocessing is converting the spike stream input from the DVS sensor into processable frames. The general schematic of this method is shown in Figure 3.2. DVS sensor data is in the form of a spike stream. In other words, the data from this sensor consists of consecutive packets, each containing spike information. The most important information for us is the spike coordinates (x and y of the spike in the image coordinates) and the spike polarity. The DVS sensor operates such that when the light intensity of any sensor point changes, a spike is produced that includes the direction of these changes (positive or negative) and its coordinates. Therefore, if the image is static and has no change, no spike is produced, and consequently only those points of the image that have changes produce spikes.

To convert the spike stream to frames, we use Algorithm 1. In this algorithm, it is assumed that the spike stream is temporally sorted, although in practice this is not the case. In general, the operation method of this algorithm is as follows: we start processing the spikes in order. Then, by considering the frame-length constant, we determine the duration of each frame. Then we partition the spikes into intervals of frame-length duration. For this purpose, we use Algorithm 1. As a result, the frame-set variable is a set of spike stream partitions, each representing the spikes of each frame.

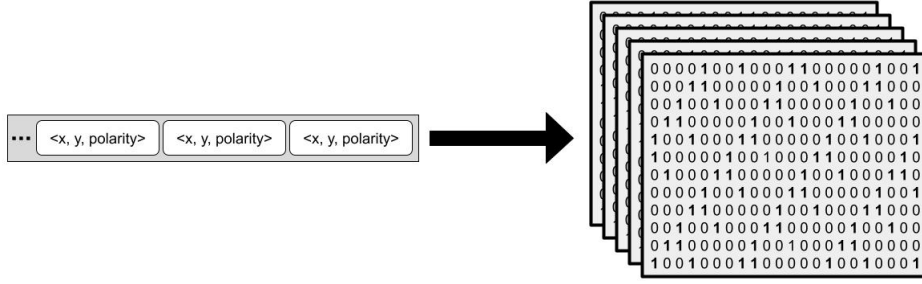


Figure 3.2: As can be seen, the input data is in the form of a train of spike packets that are continuously injected into the system. Each of these packets contains the coordinates and polarity of spikes. The algorithm output is also a set of spike images in which the value of each element is either zero or equal to the spike polarity at that point.

---

**Algorithm 1:** Converting spike stream to frame-based image

---

**Result:** Framed dataset.

```

frameSet  $\leftarrow \emptyset$ ;
frameStartTime  $\leftarrow$  Time of first spike in system;
// Assuming spikes are sorted in time.
while There is spike to process do
    currentTime  $< -$ frameStartTime;
    while currentTime  $\leq$  frameStartTime + frameLength do
        spike  $\leftarrow$  getNextSpike();
        frameSet  $\leftarrow$  frameSet + {spike};
        currentTime  $\leftarrow$  spikeTimeStamp;
    end
    frameStartTime  $\leftarrow$  currentTime;
end

```

---

In the next step, we convert each of these partitions into a frame. For this purpose, we consider an empty frame (a zero matrix with the dimensions of the image). Then it is sufficient to process the spikes and increase or decrease the value of frame points where spikes have occurred.

For data framing, there are two important parameters: the number of frames and the frame length. The frame length was explained in the previous algorithm. This parameter represents the duration we wait for image spikes to arrive and enter them into the frame. The second parameter is the number of frames. This parameter represents the number of frames that the dataset has.

An important point in cutting the spike stream is paying attention to the temporal

labels of the data. In this dataset, data labels are stored in tabular form indicating the start and end of each gesture in a file. To cut the data, we must ensure that one data partition does not contain two different gestures. To solve this problem, when generating partitions, when we examine and concatenate spikes, we pay attention to the time the spike occurred. Based on the spike occurrence time, its label can be determined. If the label of the spike under examination equals the label of the first spike of the partition, this means that both spikes belong to one gesture. Therefore, we add both to the partition, and if their labels are different, we discard all spikes of the partition and start generating a new partition. The reason for this is also that the length of this partition will be less than the desired value, and it cannot be used in the dataset.

The next step, which is one of the most important steps after data preprocessing, is that after framing the data, we save them in files in NumPy file format. The reason for this is that this part of preprocessing is very time-consuming and there is no need to perform it at each stage. In this thesis, data with lengths of 20, 80, and 120 frames have been examined, with each frame length being approximately 50 microseconds.

The polarity or direction of changes of each spike can be positive or negative. But biological spikes are only positive, and consequently in our model, only positive spikes are produced. Therefore, we need a more precise definition of the network inputs. We can disregard the polarity of inputs or have an interpretation for negative inputs. In this thesis, we propose four implementation methods for this issue:

1- Ignoring spike polarity: The simplest method for processing spikes is to disregard their polarity. In other words, for the network, it is only important that there are changes at the desired point, and the direction of changes is not important. Processing in this way can be considered equivalent to processing image edges. In such processing, the gradient direction of edges is not important and only the edges themselves have value. To implement this method, we disregard the polarity of spikes and at any point of the image where there is a spike, we consider the value 1 and other points 0.

2- Single channel method: In this method, like the previous method, we consider the frame plane as zero and place the spikes on it. The only difference in this method is that we set the spike values to their polarity value. That is, we set positive spikes to positive one and negative spikes to negative one. This method has no difference from the previous

method in terms of the number and volume of input data, and the only difference is the possibility of these spikes being negative. A point that makes this method not a desirable method for framing spike streams is that in this method we have negative spikes that do not exist in the real world and in biological neurons. Therefore, it moves the model away from similarity to brain neurons.

3- Dual channel method: Like the first method, in this method all spikes have a positive value of one and all other points have a value of zero. The only difference in this method is that two separate layers are considered in it. The first layer is for spikes with positive polarity and the second layer is for spikes with negative polarity. The reason for this is that spikes are separated in terms of polarity, on the other hand, we will not have a spike with a value of negative one in the system. Also, in this method, the volume of input data doubles.

4- Spike accumulation method: The problem with all the above methods is that they do not consider the number of spikes at a point and only examine the presence and absence of spikes. For example, if there is more than one spike at a point, spikes from the second onwards are disregarded. For this reason, a large volume of information is lost at this stage. To solve this problem, we consider the frame such that instead of showing spikes, it shows the spike integral over the frame time interval. In this way, frame values are no longer zero and one and can have positive or negative real values. On the other hand, this method in computer simulations is more similar to the real spiking model.

### 3.1.3 Data Augmentation

The next step in data preprocessing is generating and adding new data based on existing data to the dataset and augmenting it. Although this step is optional, performing this work will strengthen the dataset. In other words, by doing this, the number of trainable data in the dataset increases, and as a result, the network can be trained better and better results can be achieved.

For generating image data, there are various methods that cannot all be applied to spike images. For example, in pixel-based images, color change, cropping, or resizing the image, etc. can be used. But these methods cannot be used in spike images. For example, cropping the image in a spike frame is not possible because the cropped frame cannot be

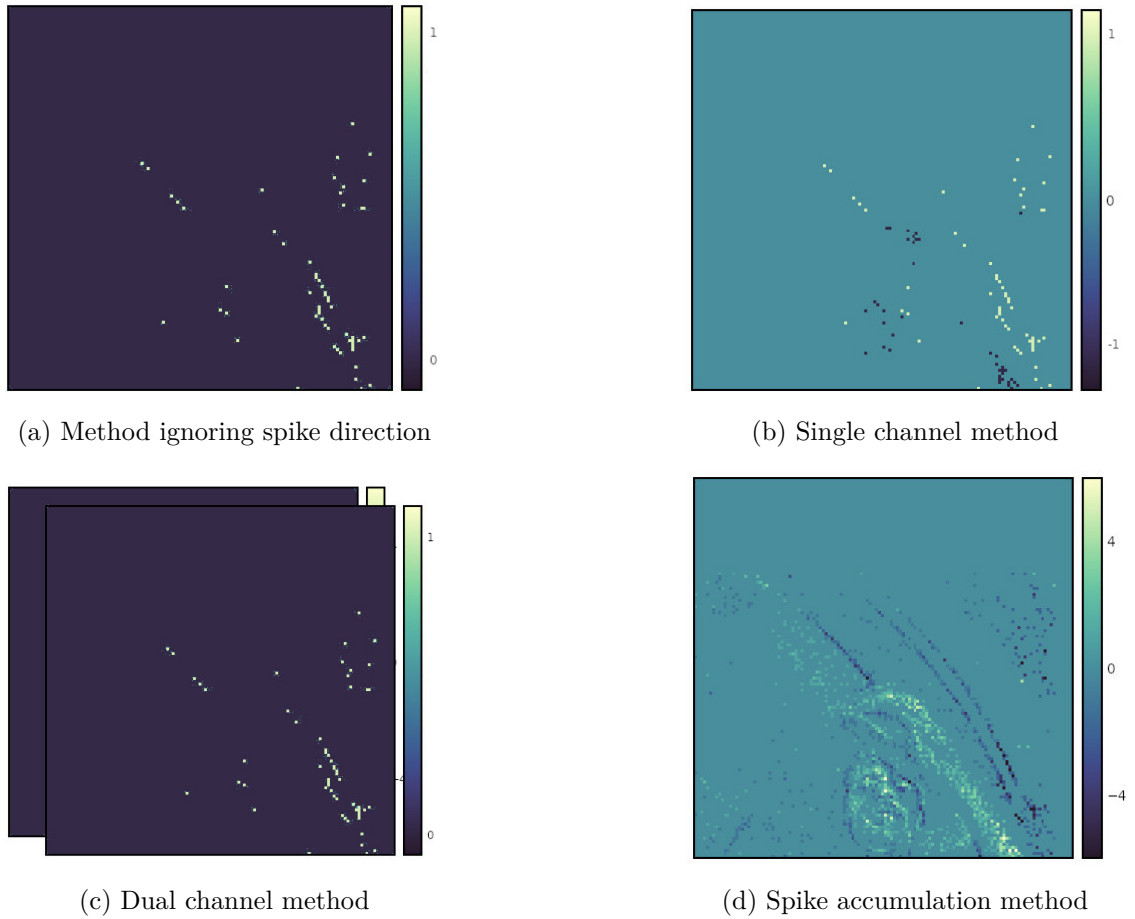


Figure 3.3: Images of a cut time frame in different framing methods are shown. Figure 3.3a shows the method ignoring polarity. Figure 3.3b shows the single channel method. Figure 3.3c shows the dual channel method. And Figure 3.3d shows the spike accumulation method. As can be seen, in the spike accumulation method, a more tangible shape is formed.

resized.

The method we used for dataset augmentation is simply image displacement. In other words, the coordinates of each image set are added with random noise that does not change over the time of that data. This is like the desired subject having a slight displacement. Therefore, in this method, the distribution of spikes does not change. And only the frame shape changes, and we can say we have generated new data.

The final step in the data preprocessing stage is dividing them into training and test data. In this section, we randomly divided the obtained frame set into two parts: training and test. The only important point in this section is that in the test sets, which are the basis for calculating system accuracy and other metrics, no augmentation and addition method has been used. Because this data should not have random noise so that reproducible results can be obtained.

## 3.2 Network Architecture

In this section, we examine the layers and techniques used in the network architecture. These techniques include various layers implemented in the network as well as techniques to improve network performance. But before that, a noteworthy point should be mentioned. One of our goals regarding the use of Spiking Neural Networks is to get closer to the biological structure of the brain. As we know, architectures such as fully connected layers and convolutional layers are arranged with a specific order. This is while biological neural networks have a complex and intertwined structure that has emerged evolutionarily. Due to this complexity, simulating these structures on GPUs is very difficult and requires very high processing power. While structured architectures have simpler implementation and also have higher processing speed. Also, due to the simpler structure, training these networks is also simpler, because training signals in these networks have a simpler mathematical model. For this purpose, in this thesis, and in most similar research and articles, these structures have been used.

### 3.2.1 Dense Layer

The first and most important layer in neural networks is the dense layer. As we know, this layer consists of connecting all neurons to all neurons of the previous layer, or more precisely, from the complete connection of all synapses to all inputs. The operation method of this layer is shown in Algorithm 2.

---

**Algorithm 2:** Processing spiking dense layers

---

```

// this has the shape of (batch, nb-frames, input-shape)
 $x \leftarrow inputSpikes;$ 
 $y \leftarrow 0;$ 
 $syn \leftarrow 0;$ 
 $mem \leftarrow 0;$ 
 $spk \leftarrow 0;$ 
for  $i = 0$  to Number Of Frames do
     $h \leftarrow x[i].W;$ 
     $rst \leftarrow spk;$ 
     $syn \leftarrow \alpha * syn + h;$ 
     $mem \leftarrow \beta * mem + syn - rst;$ 
     $spk \leftarrow heavySide(mem);$ 
     $y[i] = spk;$ 
end
return  $y;$ 

```

---

This algorithm is executed for each data and simulates the neurons of the current layer. It should be noted that as mentioned earlier, for simplicity and speed of network simulation, we perform it layer by layer. Therefore, we start simulating from the first layer, and after running the layer for all times, we move to the next layer. Therefore, in this layer, the input value, or  $x$ , also has a time dimension. To process the layer, we traverse in this dimension. As can be seen, the variable  $i$  traverses in the time dimension to simulate the layer.

Then the input at moment  $i$  is multiplied by the network weights and forms the input current value, which is stored in the variable  $h$ .  $h$  is actually the same variable observed in Artificial Neural Networks. In the next step, the value of input current to the neuron is calculated. This value is stored in the variable  $syn$ . Then the internal potential of the neuron called  $mem$  is calculated. This value increases using  $syn$  and decreases using

rst. Also, the coefficient beta decreases over time. Finally, this value is filtered using the heavy-side function and values greater than one lead to a spike.

Of course, two other mechanisms, weight normalization and not using the current weight coefficient, cause changes in the above algorithm, which we will discuss later.

### 3.2.2 Convolutional Layer

The implementation of neurons in this layer is similar to neurons in the dense layer, and the only difference is in how they receive input. In other words, it can be said that the operation method of the potential mechanism and neuron spiking in this layer is completely similar to the dense layer. Because both use Leaky Integrate-and-Fire neurons. The only difference between these two layers is in input processing, which has no difference from other Artificial Neural Networks.

In this section, we first form the kernel weights, then slide them over the input data to obtain the observed data or h. The only noteworthy point of these layers is in their input. Unlike dense layers that expect input in the form [number of input data, simulation time, batch size], they receive input in the form [image width, image length, simulation time, image channels, batch size]<sup>3</sup>.

In this project, these layers have been implemented in one, two, and three-dimensional formats. However, one-dimensional layers have no application in this problem. Also, after conducting experiments, we concluded that the most suitable type of processing for this problem is processing it in three dimensions. With the difference that the length of the kernel used in the time dimension (third dimension) is one. The reason for using three-dimensional layers is its faster processing speed and also the ability to process in the time dimension if needed.

### 3.2.3 Max-Pooling and Avg-Pooling Layers

Following the convolutional layers, pooling layers have also been implemented. These layers have been implemented using PyTorch tools and require no other changes. These layers have no trainable parameters. The Max-Pooling layer has spike output and during

---

<sup>3</sup>This order is suitable for three-dimensional convolutional layers.

the neuron processing interval, if it sees any input with a spike, it also produces a spike. But the Avg-Pooling layer loses its spiking property because it removes data from the zero-one state and assigns real number values. For this reason, we did not use this layer in our networks, and its implementation was merely for examination and testing.

### 3.2.4 Not Using Synaptic Current Coefficient ( $\alpha$ )

After conducting numerous experiments, we concluded that to simplify the network model and also to make it faster, we do not use synaptic weights or  $\alpha$ . The insight behind this decision is that the network can optimize its weights using the  $\beta$  parameter, and in fact, softening the input current will not have an added advantage for the neuron. For this reason, lines 9 and 10 of Algorithm 2, which we referred to earlier, can be rewritten as follows:

---

**Algorithm 3:** Dense layer processing without synaptic constant

---

```

x ← inputSpikes;
y ← 0;
syn ← 0;
mem ← 0;
spk ← 0;
for i = 0 to Number of Frames do
    h ← x[i].W;
    rst ← spk;
    mem = (mem − rst) * beta + h * (1 − beta);
    spk ← heavySide(mem);
    y[i] = spk;
end
return y;

```

---

In this way, we no longer need the *syn* variable, and the *h* value will directly affect *mem*. The only point is that the *h* value is controlled using the coefficient 1-*beta* so that the *mem* value does not exceed one.

### 3.2.5 Normalizing Output Spikes

One of the techniques that leads to better training of neural networks is the use of Batch Normalization layers or Batch-Norm. Using this layer in Artificial Neural Networks

causes a significant increase in network training accuracy. The reason for this is that input data after being broken into smaller batches find a different distribution compared to the original data. Sometimes this distribution can be so different that it causes problems in network training. Also, in higher layers, the network layer output can be very different, which causes too much decrease or increase in the training signal, which causes the network to not train. Using normalization layers causes the output of each layer to be in an expected normal range, and as a result, the next layer can train more easily and with greater accuracy.

The conventional normalization method or use of Batch-Norm layers is not directly applicable to Spiking Neural Networks. Because these layers produce their output values continuously and in real numbers. In other words, the normalization operation converts spiking variables to real variables. To solve this problem, we implemented the normalization operation inside the neuron. The implementation of this method can be seen in Algorithm 4. This algorithm is similar to Algorithm 3, and the only difference is in calculating the threshold value or *mt*hr. This value is a normalized value of the neuron potential. And in this algorithm, instead of calculating spikes from the potential value, spikes are calculated from the normalized value of *mt*hr.

---

**Algorithm 4:** Dense layer processing with Batch Normalization

---

```

nrom := ||w||2;
x ← inputSpikes;
y ← 0;
syn ← 0;
mem ← 0;
spk ← 0;
for i = 0 to Number of Frames do
    h ← x[i].W;
    rst ← spk * b * nrom;
    syn ← alpha * syn + h;
    mem ← beta * mem + syn - rst;
    mthr ← mem/norm + b;
    spk ← heavySide(mthr);
    y[i] = spk;
end
return y;

```

---

In this algorithm, in addition to the training weights  $w$  and  $\beta$ , another variable is also trained, which is called the normalization variable or  $b$ , which can change between zero and one. In this algorithm, the reset term,  $rst$ , instead of being directly replaced with the spike values of the past cycle, is multiplied by  $b$  and the L2 norm of the weights. Therefore, it finds a softer value compared to the past, and based on whether the number of spikes was low or high in the past cycle, the potential value of this layer cycle is controlled, and as a result, it can be said that neuron potentials will always be in reasonable ranges.

### 3.2.6 Initial Weights

Determining initial weights in Spiking Neural Networks is of great importance, and their incorrect determination can cause a decrease in network accuracy or even make it untrainable. The reason for this issue is that the training error calculated in these networks is calculated using the network's output spikes. And if the synapse weights of the neuron are low, the neuron may not produce any spikes, and in this case, no training signal is produced and the network cannot do anything to correct its weights. Also, if the random range of weights is large, it causes neurons with lower potential to be eliminated in the normalization operation and not be trained.

In the experiments and implementations conducted, we concluded that weights should have a mean of zero and a standard deviation between 0.001 to 0.01 for convolutional layers and a value of 0.4 to 0.6 for dense layers, which produces the best result and causes the network to reach higher training accuracy at the appropriate time. It should be noted that network weight values are assigned using normal distribution, and the standard deviation value of this distribution is obtained from equations 3.2 and 3.3 for convolutional layers, and 3.4 and 3.5 for dense layers. The value  $std$  in this equation is the same standard deviation value that was explained earlier.

$$std_w = std \cdot \sqrt{\frac{1}{c \cdot \prod k_i}} \quad (3.2)$$

$$std_v = std \cdot \sqrt{\frac{1}{c}} \quad (3.3)$$

In this equation,  $std_w$  is the standard deviation of the normal distribution related to network weights (convolutional weights), and  $std_v$  is related to recurrent weights (if any).

The value  $c$  is the number of output channels of the layer, and the value  $k_i$  shows the kernel size in the  $i$ -th dimension.

$$std_w = std. \sqrt{\frac{1}{L^{i-1}}} \quad (3.4)$$

$$std_v = std. \sqrt{\frac{1}{L}} \quad (3.5)$$

In this equation,  $L$  specifies the number of neurons in the current layer or in other words the number of layer outputs. And the value  $L^{i-1}$  specifies the number of neurons in the previous layer or the same number of layer inputs.

### 3.2.7 Dropout Layer

Another idea for optimizing the network and increasing its performance is adding Dropout layers. The main idea of Dropout layers is that in the training stage, neurons are randomly excluded from the backpropagation operation and optimization, and this increases network accuracy. The interpretation of this work is that by doing this, we actually have a large number of different and parallel neural networks working together in the network simultaneously. In our implementation, we used Dropout layers available in standard machine learning libraries.

### 3.2.8 Lateral Connections

The idea of using lateral connections is not a new idea, and in the past, networks have been presented that have benefited from these connections. In general, these connections can help the network to be able to produce similar results with fewer layers and have similar power. Lateral connections are connections between neurons of each layer, as a result, the neuron input current equation changes to equation 3.6.

$$a_i^m = \sum_j W_{ij}^{m-1} z_j^{m-1} + \sum_{k \neq i} V_{ik}^m z_k^m \quad (3.6)$$

In this equation,  $W$  is the network weights and  $V$  is the weights between neurons or the same lateral weights. Also, to implement these connections in the neuron algorithm, the algorithm must be changed as follows. First, we consider the variable  $d$  as follows:

$$d = W.W^T \quad (3.7)$$

Then to calculate the rst value, we multiply the spike values of the previous cycle by  $d$ .

### 3.2.9 Recurrent Connections

Recurrent connections allow us to implement networks with recurrent architecture. Regardless of the layer type, recurrent connection capability has been implemented in all neurons, therefore this architecture can be implemented with dense layers and convolutional layers simultaneously. The implementation of these layers is such that in each neuron processing cycle, we consider the layer spikes in the previous time cycle and add it to the current time neuron input. To understand this, consider equation 3.8.

$$h(t) = I(t) + S(t - 1).V \quad (3.8)$$

In this equation,  $I$  is the neuron input from the previous layer,  $S$  is the neuron output spikes,  $V$  is the weights related to recurrent relationships, and  $h$  is the input current to the neuron potential relationship.

### 3.2.10 Using Winner-Takes-All Mechanism

One of the mechanisms used in the network decision layer is the Winner-Takes-All mechanism. This mechanism briefly operates such that in the decision layer, if the potential of a neuron reaches its tolerance threshold and produces a spike, the potential of its neighboring neurons also decreases. In other words, the neuron that produces a spike earlier receives the training signal. Therefore, neurons compete with each other to spike earlier. On the other hand, using this mechanism causes a general reduction in produced spikes and as a result, makes the output of this layer sparse.

## 3.3 Decision Making and Loss Function Calculation

In this section, we examine the implemented structures for the decision layer and error calculation as well as for optimization.

In general, we implemented this network with two perspectives. The first perspective is that we partition the data stream and treat it as a labeled dataset. And the second perspective is that we train the network on the data stream, similar to reinforcement learning methods. Each of these methods has its own advantages and disadvantages. The first perspective seems to be a very suitable method for processing data streams, but in return, it introduces difficult issues into the system. Perhaps the most important problem of this method is how to implement backpropagation methods, and also the inability to simulate neurons layer by layer, which made this method very slow, and finally we used the second perspective to solve this problem.

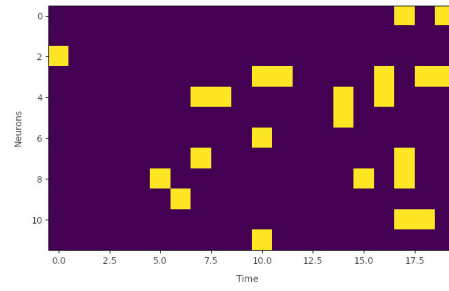
The second perspective is a method in which we partition the data into equal parts so that each data has a specific number of time frames. This method is actually the same method that was explained in previous sections for data preparation. The advantage of this method is that it is simpler for simulation on GPUs. And in practice, in the experiments conducted, this method has shown better performance.

### 3.3.1 Decision Making in the Output Layer

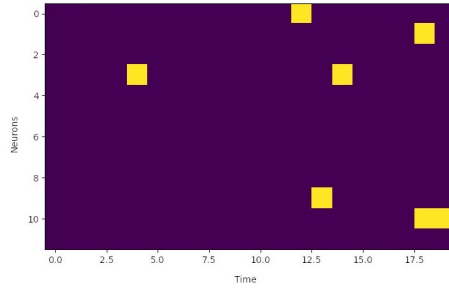
A point that exists in calculating the loss function is that calculating the output and extracting it from the network is not obvious. In other words, in the last layer of the spiking network, there is a dense layer in which the potential values of its neurons change over time. To determine a specific class from the output values of these neurons, multiple methods can be defined. The methods implemented in this thesis are:

#### **Spike-based Methods**

These methods themselves include two parts. The first method is spike count and the second method is first spike time. In the first method, the output class is the class belonging to the neuron that had the most spike count, and the second method, the output belongs to the neuron that produced the first spike. After investigations, we concluded that the problem of the first method is excessive spike production, which causes loss of network optimization and accuracy. And the second method has shown better performance because the network tries to produce fewer spikes. But on the other hand, the problem



(a) Spike count method



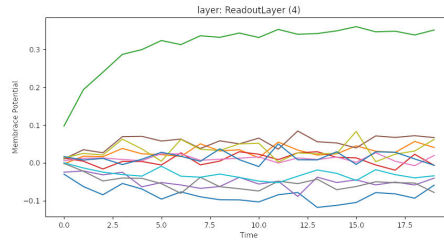
(b) First spike time method

Figure 3.4: In this diagram, we observe the activity of neurons in the last layer of the network. The top figure is related to the case where the network has been evaluated and trained based on the number of spikes produced. The bottom figure is related to the network in a state where it has been trained based on the class of the first spike produced.

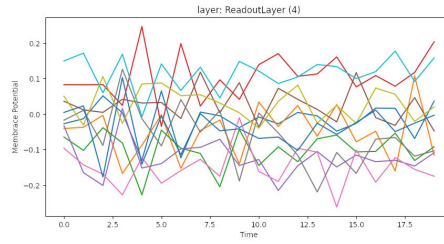
of this method is that spikes are produced with more delay. This same issue causes the training process to become slower.

### Potential-based Methods

These methods are based on not needing to count output spikes and by examining neuron potential values, selecting the output class. In this category, two methods have also been implemented. The first method, calculating the average potential of neurons and selecting the neuron with the highest average, and the second method, calculating the maximum potential of each neuron (during the network simulation period on the data) and selecting the neuron with the highest maximum potential. Between these two methods, the second method has also shown better performance because this method applies more pressure (more error) on other neurons so that they do not have high values at any point, but in the first method, it was only sufficient for other neurons to have low values at most points and they could sometimes take high values.



(a) Maximum potential method



(b) Average potential method

Figure 3.5: In this diagram, the potential of neurons over time after training is shown. The top figure shows the output layer with maximum decision maker and the bottom figure shows the same layer with average decision maker. As can be seen, the maximum decision maker applies more pressure to keep the potential low.

### 3.3.2 Loss Function Calculation

After determining the output type, we must calculate the network error value. As mentioned earlier, the first step in this area is defining the loss function or cost function. Since our goal in this thesis is to produce a network with classification capability, the Negative Log-Likelihood Loss function has been selected as the cost function. In this function, we try to enter the difference between the output value and the desired value, both of which are between zero and one, with logarithmic slope into the error value. This value is shown in equation 3.9.

$$L = -\frac{1}{n} \sum_x \sum_{k=1}^K y_k \log(a_k) \quad (3.9)$$

In this equation,  $x$  is the input data on which we calculate the cost function value.  $K$  is the total number of problem classes, and  $y_k$  is the specified value for the  $k$ -th class in the dataset. The value  $a$  also represents the model output value. It should be noted that the value  $y_k$  can be zero or one, and the value  $a$  can take a value between zero and one. Therefore, in this formula, in fact, for each data from the dataset, we will only calculate

one value  $\log(a_k)$  (the value corresponding to the related class), and other values are not included in the cost function because their  $y_k$  value is zero. Next, to optimize network weights, we use equation 3.10.

$$W_{ij} \leftarrow W_{ij} - \eta \frac{\partial L}{\partial W_{ij}} \quad (3.10)$$

Then for output layer weights we will have:

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial O_i} \frac{\partial O_i}{\partial V_i} \sum_t \frac{\partial V_i}{\partial U_i(t)} \frac{\partial U_i(t)}{\partial W_{ij}} = (O_i - 1) \sum_t \frac{\partial U_i(t)}{\partial W_{ij}} \quad (3.11)$$

In this equation, to calculate  $\frac{\partial U_i(t)}{\partial W_{ij}^b}$ , we use equations 3.14 to 3.16. In subsequent layers, we denote the transferred error with  $\delta$ . Therefore, we will have:

$$\frac{\partial U_i(t)}{\partial W_{ij}} = \sum_t \frac{\partial L}{\partial S_i(t)} \frac{\partial S_i(t)}{\partial W_{ij}} = \sum_t \delta_i^h \frac{\partial S_i(t)}{\partial W_{ij}} \quad (3.12)$$

$$\delta_i^o = (O_i - 1) \quad (3.13)$$

In equation 3.12, to calculate  $\frac{\partial S_i}{\partial W_{ij}}$ , we can use the following equation.

$$\frac{\partial S_i(t)}{\partial W_{ij}} = \Theta'(U_i(t)) \frac{\partial U_i(t)}{\partial W_{ij}} \quad (3.14)$$

$$\frac{\partial U_i(t)}{\partial W_{ij}} = \beta \frac{\partial U_i(t-1)}{W_{ij}} + \frac{\partial I_i(t-1)}{W_{ij}} \quad (3.15)$$

$$\frac{\partial I_i(t)}{W_{ij}} = \alpha \frac{I_i(t-1)}{W_{ij}} + S_j[t-1] \quad (3.16)$$

The only remaining value in this equation is the value  $\Theta(U)$ . As mentioned earlier, the derivative of the  $\Theta$  function is not calculable because the Heaviside function is discontinuous at point zero and has zero derivative at other points. To solve this issue, the surrogate gradient method presented in Neftci et al. (2019) can be used.

## 3.4 Implementation

As mentioned earlier, the implementation of this thesis was done in Python 3 and on the PyTorch platform. The reason for using Python was the existence of numerous libraries in the fields of data science, data processing, and image processing, and the reason for using the PyTorch platform was to benefit from tools related to neural networks and machine learning, including automatic gradient calculation capabilities of functions, optimization tools such as Adam.

To examine the implementation, it is better to look at the file structuring of the project. As can be seen in Figure 3.6, the project has a main folder called `scnn` in which files related to network implementation exist. Alongside it, the `navgesture` and `ibm-data` folders are located, which are the location for placing data interface implementation files (user interface, related to working with their corresponding datasets). In the `tools` folder, side tools related to software execution are placed. Finally, two folders `logs` and `results` will also be completed after running the network, which contain network output files. Files are also located in the main project folder, the most important of which are `main-code.py` and `main-code.ipynb`, which are respectively the project execution file in Python format and Jupyter Notebook format.

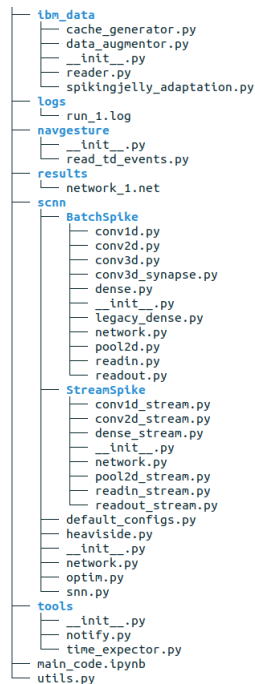


Figure 3.6: Schematic of project folders and files.

In the `senn` folder, two subfolders `BatchSpike` and `StreamSpike` are located, which are similar implementations of the spiking network, with the difference that `BatchSpike` is for batch processing and the other is for stream processing of data. Batch processing is more suitable for increasing network performance, while stream processing has much more similarity to brain structure. As mentioned in previous sections, in GPU simulation, batch processing has higher efficiency. For this reason, in this thesis, to be able to use the maximum available processing power, we focused more on batch processing. In the following, we review a summary of the classes implemented in these folders.

The architecture used in this implementation is such that the user can easily generate different networks and can easily add customized layers to their network. For this purpose, a base class for layers called `neuron-base` is used, and all implemented layers inherit from this class. As a result, any class that inherits from this class can be added to the network. This abstract class is a base model of a layer of spiking neurons. One of the most important parameters of this class is the number of input and output neurons of the layer, the number of its input and output channels (for convolutional neurons), and the type of spike function. Also, Dropout, Weight-Decay mechanisms, and recurrent connections, due to having the same implementation for all types of layers, have been implemented in this class.

To implement and add a new layer to the network, it is only sufficient to inherit from the `neuron-base` class and implement the following functions in the desired layer. Also, three parameters `IS-CONV`, `IS-SPIKING`, and `HAS-PARAM` must also be assigned and specified. The first parameter indicates whether the layer is convolutional. With the help of this parameter, the network automatically reduces data to one dimension between convolutional and dense layers<sup>4</sup>. The second parameter indicates whether the layer is spiking, and finally, the last parameter indicates whether the layer needs training or not. After specifying the parameters, it is sufficient to implement the following functions to be able to load the layer into the network:

- `__init__`: This function, which is the constructor function in Python and is related to class structuring. In this function, the `__init__` function from the parent class must be called. After that, user input values as well as neuron and channel number values can be used.

---

<sup>4</sup>In machine learning libraries such as PyTorch, this operator is generally known as **Flatten**

- forward-function: The main function of the neuron model of each layer. This function is called once for each layer every time the network runs. In this function, equations related to executing and processing neurons must be implemented.
- train-able: This function returns a list of trainable weights so that the network class can identify and train these parameters.
- `__str__`: This function returns the layer schema in written form.
- reset-parameters: This function assigns layer weights with specified random values.
- draw: This function is called to draw diagrams related to network and neuron analysis.

In the next stage, two layers readin and readout are located. The readin layer has the role of shaping input data so that it has similar parameters to other layers. The readout layer has the role of interpreting the network output. The operation method of this layer is explained in detail in section [3.3.1](#).

The classes dense and legacy-dense are both implementations of the dense layer, and the only difference between them is in implementation. The use of synaptic weight mechanism in the dense layer has been disregarded because in the experiments conducted, we showed that this mechanism has not had much effect on network performance.

In this implementation, one, two, and three-dimensional convolutional layers have been used, each of which has inherited from its previous layer. The distinguishing feature of these classes is in their forward-function. Although this function operates similarly for all three types, and their difference is in their matrix multiplications and their convolution operator type. In these classes, for the convolution operator, functions available in PyTorch have been used. The reason for this is the sameness of this operator in spiking networks and non-spiking networks.

In this folder, all classes except the network class are related to layer implementation. The network class, which has inherited from the `torch.nn.Module` module located in PyTorch, has the duty of storing and maintaining network layers and calling their functions when needed. Helper functions have been implemented for this purpose. For example, the predict function that executes all network layers in order and returns the output of the

last layer. Another important duty of this class is implementing the fit function. Calling this function leads to network training. This function first performs the necessary settings for network training, then repeats the layer training operation for the required number of epochs.

The surrogate gradient method implementation has been done in the heavyside class. For this purpose, we implemented the Heaviside function (step function) by inheriting from the autograd class or automatic gradient, available in the PyTorch library. This class works like a hard spike function, with the difference that when called, it stores input values and keeps neurons that have produced spikes in memory. Then by rewriting the derivative function (here the backward function), it calculates the layer value derivatives using spikes that have been produced during past times.

The last class from the main code set is the SNN class. This class is considered the network user interface and is the point at which the user interacts with the network. This class includes functions for adding different layers and building the network, functions for storing and retrieving networks from memory, as well as functions for drawing required diagrams for network analysis. In practice, this class is a wrapper for the Network class.

Alongside the scnn folder, two other folders exist for working with datasets, the ibm-data folder and the navgesture folder. The first folder is for working with the IBM-DVSGesture dataset and the navgesture folder is for working with the NavGesture dataset. It should be noted that both of these folders contain the same function called load-all located in the `__init__.py` file, which has the same input and output. Therefore, the user can use them without needing to know how these modules work. Both modules have the duty of reading data from disk, preprocessing them, and loading them into RAM memory for network use. The ibm-data module has two implementations: implementation based on the SpikingJelly library and native implementation. The SpikingJelly implementation is related to method [3.1.2](#), and the native implementation is related to the other three methods.

## 3.5 Results

In the first step, we examine the network neurons and ensure that the network works correctly. For this purpose, we first form a small network consisting of two layers, the first layer containing 20 spiking neurons and the second layer being the decision layer containing two neurons. In this example, the input is in the form of a small dataset of spike frame-based images, each containing 10 frames. And the images of this dataset are divided into two categories: spikes on the main diagonal and spikes on the secondary diagonal of the image. The activity of the first and second layer neurons of this network, before and after training, is observed in Figure 3.7. As can be seen, this network can easily distinguish these images after training.

In the next stage, it is time to perform the same experiment for convolutional layers. The only necessary change for this is replacing the first layer of the previous network with a convolutional layer, and then re-running the network. These layers are also easily capable of learning this dataset.

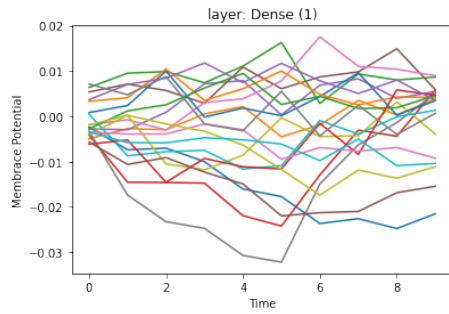
### 3.5.1 Complete Network Testing

In the next step, we trained a complete network with one layer with 128 spiking neurons and one decision layer using the IBM-DVSGesture dataset. Data with a length of 20 frames was extracted from the dataset and trained in batches of 16. The spike polarity extraction method was also the spike direction accumulation method. Also, in this experiment, the network was trained in 40 training cycles with a learning coefficient of  $10^{-4}$ . As observed in Figure 3.8, this network was able to correctly recognize training data with 100

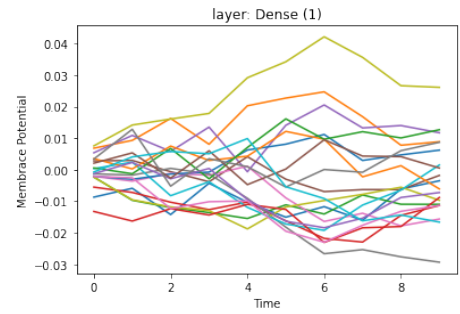
### 3.5.2 Frame-Based Image Generation Methods Testing

In this section, we examine the methods of extracting and partitioning spikes from the spike stream and generating frame-based images. These methods, as examined in previous sections, include the following methods:

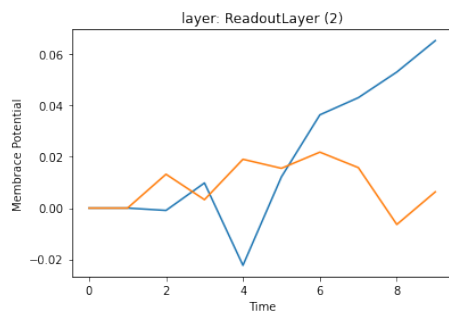
- Method ignoring spike polarity
- Single channel method (storing spike polarity value in one image channel)



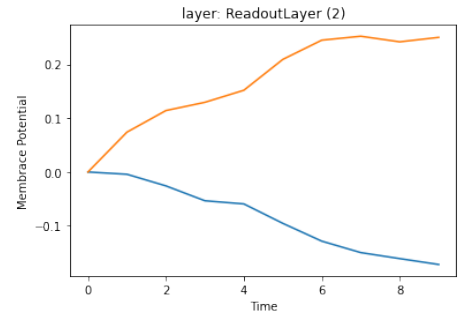
(a) First layer before training



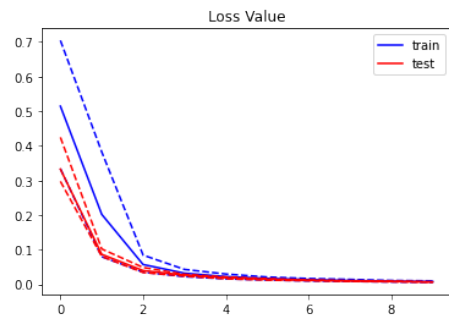
(b) First layer after training



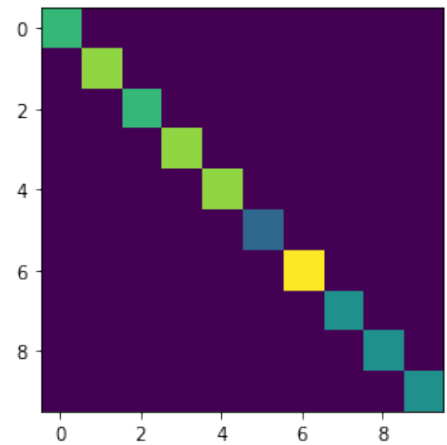
(c) Decision layer before training



(d) Decision layer after training

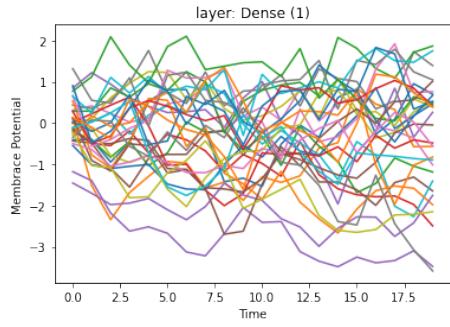


(e) Loss function diagram during training process

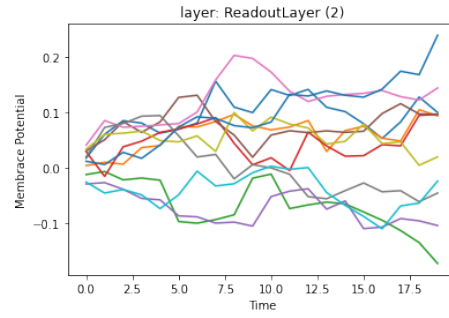


(f) Example of training data

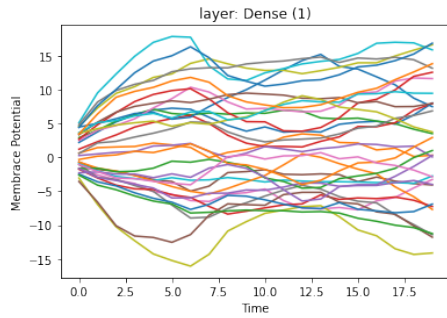
Figure 3.7: In the first two figures, the potential of a number of random neurons from the first layer is observed, and as can be seen, trained neurons have more differentiated behavior. In the next two figures, we also observe the same thing regarding neurons of the decision layer.



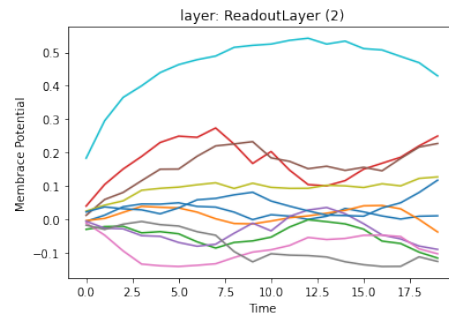
(a) First layer before training



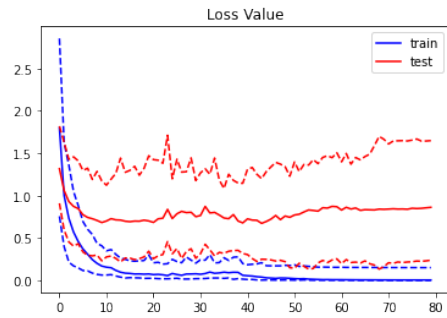
(b) First layer after training



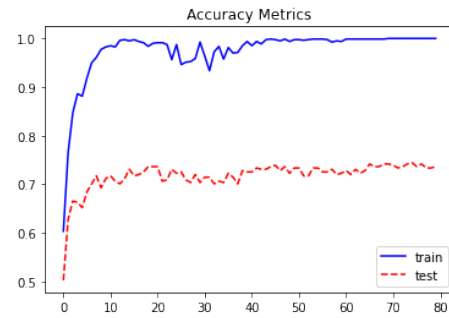
(c) Decision layer before training



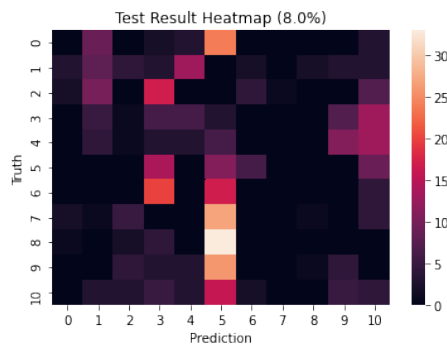
(d) Decision layer after training



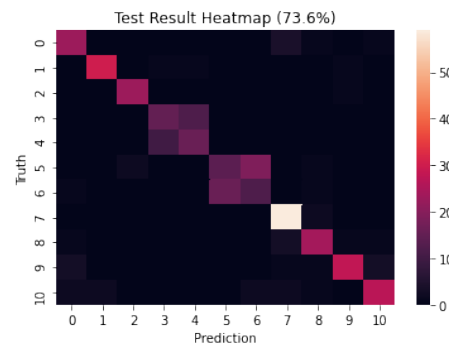
(e) Loss function diagram during training process



(f) Network accuracy percentage diagram during training process



(g) Confusion matrix of answers before training



(h) Confusion matrix of answers after training

Figure 3.8: In this diagram, we observe the potential of network layers before and after training. Also, based on Figures 3.8g and 3.8h, we can conclude that the network has the ability to recognize correct classes to an acceptable extent.

- Dual channel method (storing spikes in two separate layers based on their polarity)
- Accumulation method (summing spike polarity values)

In this experiment, four different networks were used, including two dense networks and two convolutional networks. By doing this, these four methods are examined in both simple and convolutional architectures. It should be noted that this experiment was performed on the NavGesture dataset and only 30 training data files were used instead of the entire dataset. As can be seen in Table 3.1, the polarity accumulation method shows better performance. The reason for this performance increase is probably that in other methods, some information is lost. Because spikes do not accumulate on top of each other.

| Network   | Ignoring Polarity | Single Channel | Dual Channel | Polarity Accumulation |
|-----------|-------------------|----------------|--------------|-----------------------|
| Dense.128 | 51%               | 50%            | 70%          | 68%                   |
| Dense.512 | 80%               | 84%            | 83%          | 87%                   |
| CNN.64-64 | 82%               | 83%            | 84%          | 96%                   |
| CNN+Dense | 79%               | 83%            | 81%          | 97%                   |

Table 3.1: Results of frame-based image generation method experiments

### 3.5.3 Data Time Length Testing

In this experiment, the time length of data and their effects on network performance have been examined. Like the previous experiment, in this experiment, networks with similar parameters were used, and the only difference in this experiment is that both datasets were used.

| Network   | IBM-DVSGesture |           |            | NavGesture |           |            |
|-----------|----------------|-----------|------------|------------|-----------|------------|
|           | 20 frames      | 80 frames | 128 frames | 20 frames  | 80 frames | 128 frames |
| Dense.128 | 78%            | 75%       | 78%        | 68%        | 86%       | 84%        |
| Dense.256 | 71%            | 75%       | 75%        | 87%        | 88%       | 86%        |

Table 3.2: Results of frame length experiments

### 3.5.4 Output Type Testing

In this experiment, types of decision layers have been compared with each other. To conduct this experiment, we used networks with similar parameters and from the IBM-

DVSGesture dataset. The only difference in these networks is in the type of their decision layer. As can be seen, potential-based decision making states have remarkable performance compared to spike-based decision making states, and also decision making based on maximum potential has better performance compared to other states.

| Network   | Average Potential | Maximum Potential | Spike Count | First Spike Time |
|-----------|-------------------|-------------------|-------------|------------------|
| Dense.128 | 75%               | 78%               | 63%         | 68%              |
| Dense.512 | 74%               | 71%               | 64%         | 70%              |

Table 3.3: Results of output type experiments

### 3.5.5 Auxiliary Connections Testing

In this section, various types of recurrent and lateral connections have been compared together. This experiment was also performed only on the IBM-DVSGesture dataset. As observed in this experiment, recurrent and lateral connections do not have a significant effect on network performance and only cause an increase in processing volume and network memory consumption. However, regarding connections, more research is needed because these connections may show good performance for other problems.

| Network   | Without Aux. Connections | With Recurrent Connections | Lateral Connections | With Both |
|-----------|--------------------------|----------------------------|---------------------|-----------|
| Dense.128 | 78%                      | 71%                        | 72%                 | 72%       |
| Dense.512 | 71%                      | 77%                        | 75%                 | 77%       |

Table 3.4: Results of auxiliary connections experiments

### 3.5.6 Comparison of Spiking Network with Artificial Neural Network

Following the discussions in previous sections regarding the advantages of using Spiking Neural Networks, in this section, we have compared the results of these networks with similar Artificial Neural Networks. As can be seen in Table 3.5, Spiking Neural Networks can learn this dataset data with accuracy similar to Artificial Neural Networks. This is while on average only 2

| Network   | Spiking Neural Network |                     | Artificial Neural Network |                     |
|-----------|------------------------|---------------------|---------------------------|---------------------|
|           | Accuracy               | Floating-point Ops. | Accuracy                  | Floating-point Ops. |
| Dense.128 | 94%                    | $2.89 \times 10^4$  | 91%                       | $1.04 \times 10^6$  |
| Dense.256 | 96%                    | $5.8 \times 10^4$   | 89%                       | $2.09 \times 10^6$  |
| CNN.128   | 83%                    | $4.38 \times 10^6$  | 97%                       | $2.2 \times 10^8$   |
| CNN.32-32 | 97%                    | $2.7 \times 10^5$   | 94%                       | $1.2 \times 10^7$   |

Table 3.5: Results of comparison experiments between Spiking Neural Networks and Artificial Neural Networks

### 3.5.7 Conclusion

The main goal in this thesis is using Spiking Neural Networks to solve the dynamic gesture recognition problem. The reason for choosing these networks is also that the structure and working mechanism of neurons in these networks are more similar to our brain, and as a result, it can be said that they will have better performance with lower energy consumption. Therefore, it can be said that probably in the not-too-distant future, we will witness more widespread use of these networks in industry. As currently, many technology industries are implementing platforms for using these networks.

In this thesis, we examined these networks and their challenges, as well as examining the effect of mechanisms and tools available in these networks and their strengths and weaknesses. Also, in this thesis, we performed a complete implementation of the simulator of these networks, which is a good platform for future research in this field.

Spiking Neural Networks, although they operate slower in simulation environments and their simulation is more difficult compared to Artificial Neural Networks, regardless of their simulation overhead, these networks operate much faster than Artificial Neural Networks and their energy consumption is much lower. The main reason for this is also the use of binary weights and pulse output generation in these networks. In other words, due to the sparsity of the output pulse matrix, these networks produce fewer outputs.

Finally, as future work areas, we can mention the implementation of these networks on neuromorphic hardware. Since we did not have the opportunity to procure and use these hardware in this thesis, we were limited to using simulation in a computer environment. Using neuromorphic hardware can probably significantly affect the results of this research.

---

In addition, in my opinion, the implementations of training mechanisms used in this thesis have room for improvement. In other words, the surrogate gradient method has limitations in error transfer in networks that have many layers. Also, this method has problems when the dataset is not large enough. One of the ways that I believe can be a solution to this problem is the combined use of unsupervised training methods such as STDP as pre-training mechanisms for the network.



# References

- Amir, A., B. Taba, D. Berg, T. Melano, J. McKinstry, C. Di Nolfo, T. Nayak, A. Andreopoulos, G. Garreau, M. Mendoza, et al. (2017). A low power, fully event-based gesture recognition system. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7243–7252.
- Bellec, G., D. Salaj, A. Subramoney, R. Legenstein, and W. Maass (2018). Long short-term memory and learning-to-learn in networks of spiking neurons. *Advances in Neural Information Processing Systems* 31, 787–797.
- Bohte, S. M. (2011). Error-backpropagation in networks of fractionally predictive spiking neurons. In *International Conference on Artificial Neural Networks*, pp. 60–68. Springer.
- Bohte, S. M., J. N. Kok, and H. La Poutre (2002). Error-backpropagation in temporally encoded networks of spiking neurons. *Neurocomputing* 48(1-4), 17–37.
- Brea, J., W. Senn, and J.-P. Pfister (2013). Matching recall and storage in sequence learning with spiking neural networks. *Journal of neuroscience* 33(23), 9565–9575.
- Courbariaux, M., I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio (2016). Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1. *arXiv preprint arXiv:1602.02830*.
- DiCarlo, J. J. and D. D. Cox (2007). Untangling invariant object recognition. *Trends in cognitive sciences* 11(8), 333–341.
- DiCarlo, J. J., D. Zoccolan, and N. C. Rust (2012). How does the brain solve visual object recognition? *Neuron* 73(3), 415–434.
- Esser, S. K., P. A. Merolla, J. V. Arthur, A. S. Cassidy, R. Appuswamy, A. Andreopoulos, D. J. Berg, J. L. McKinstry, T. Melano, D. R. Barch, et al. (2016). Convolutional

- networks for fast, energy-efficient neuromorphic computing. *Proceedings of the national academy of sciences* 113(41), 11441–11446.
- Gardner, B., I. Sporea, and A. Grüning (2015). Learning spatiotemporally encoded pattern transformations in structured spiking neural networks. *Neural computation* 27(12), 2548–2586.
- Gerstner, W., W. M. Kistler, R. Naud, and L. Paninski (2014). *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press.
- Grill-Spector, K. and R. Malach (2004). The human visual cortex. *Annu. Rev. Neurosci.* 27, 649–677.
- Gyöngyössy, N. M., M. Domonkos, J. Botzheim, and P. Korondi (2019). Supervised learning with small training set for gesture recognition by spiking neural networks. In *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 2201–2206. IEEE.
- Huh, D. and T. J. Sejnowski (2017). Gradient descent for spiking neural networks. *arXiv preprint arXiv:1706.04698*.
- Hunsberger, E. and C. Eliasmith (2015). Spiking deep networks with lif neurons. *arXiv preprint arXiv:1510.08829*.
- Kheradpisheh, S. R., M. Ganjtabesh, S. J. Thorpe, and T. Masquelier (2018). Stdp-based spiking deep convolutional neural networks for object recognition. *Neural Networks* 99, 56–67.
- Kheradpisheh, S. R., M. Ghodrati, M. Ganjtabesh, and T. Masquelier (2016). Deep networks can resemble human feed-forward vision in invariant object recognition. *Scientific reports* 6, 32672.
- LeCun, Y., Y. Bengio, and G. Hinton (2015). Deep learning. *nature* 521(7553), 436–444.
- Lee, J. H., T. Delbruck, and M. Pfeiffer (2016). Training deep spiking neural networks using backpropagation. *Frontiers in neuroscience* 10, 508.

- Lillicrap, T. P., D. Cownden, D. B. Tweed, and C. J. Akerman (2016). Random synaptic feedback weights support error backpropagation for deep learning. *Nature communications* 7(1), 1–10.
- Maro, J.-M., S.-H. Ieng, and R. Benosman (2020). Event-based gesture recognition with dynamic background suppression using smartphone computational capabilities. *Frontiers in Neuroscience* 14, 275.
- Mitra, S. and T. Acharya (2007). Gesture recognition: A survey. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 37(3), 311–324.
- Mostafa, H. (2017). Supervised learning based on temporal coding in spiking neural networks. *IEEE transactions on neural networks and learning systems* 29(7), 3227–3235.
- Mostafa, H. and G. Cauwenberghs (2018). A learning framework for winner-take-all networks with stochastic synapses. *Neural computation* 30(6), 1542–1572.
- Mostafa, H., V. Ramesh, and G. Cauwenberghs (2018). Deep supervised learning using local errors. *Frontiers in neuroscience* 12, 608.
- Mozaz, M., M. Garaigordobil, L. J. G. Rothi, J. Anderson, G. P. Crucian, and K. M. Heilman (2006). Posture recognition in alzheimer’s disease. *Brain and cognition* 62(3), 241–245.
- Neftci, E. O., C. Augustine, S. Paul, and G. Detorakis (2017). Event-driven random back-propagation: Enabling neuromorphic deep learning machines. *Frontiers in neuroscience* 11, 324.
- Neftci, E. O., H. Mostafa, and F. Zenke (2019). Surrogate gradient learning in spiking neural networks. *IEEE Signal Processing Magazine* 36, 61–63.
- Neto, H. P., L. A. C. Grecco, L. A. B. Ferreira, T. C. L. Christovão, N. d. A. C. Duarte, and C. S. Oliveira (2015). Clinical analysis and baropodometric evaluation in diagnosis of abnormal foot posture: A clinical trial. *Journal of Bodywork and Movement Therapies* 19(3), 429–433.

- Nøkland, A. (2016). Direct feedback alignment provides learning in deep neural networks. *arXiv preprint arXiv:1609.01596*.
- Ren, Z., J. Meng, and J. Yuan (2011). Depth camera based hand gesture recognition and its applications in human-computer-interaction. In *2011 8th International Conference on Information, Communications & Signal Processing*, pp. 1–5. IEEE.
- Shrestha, S. B. and G. Orchard (2018). Slayer: Spike layer error reassignment in time. *Advances in neural information processing systems 31*, 1412–1421.
- Zenke, F. and S. Ganguli (2018). Superspike: Supervised learning in multilayer spiking neural networks. *Neural computation 30*(6), 1514–1541.